A Survey on Inference Optimization Techniques for Mixture of Experts Models

JIACHENG LIU, Shanghai Jiao Tong University, Shanghai, China, The Chinese University of Hong Kong, Hong Kong, China, and Hong Kong University of Science and Technology, Hong Kong, China
PENG TANG, Shanghai Jiao Tong University, Shanghai, China
WENFENG WANG, Shanghai Jiao Tong University, Shanghai, China
YUHANG REN, Shanghai Jiao Tong University, Shanghai, China
XIAOFENG HOU, Shanghai Jiao Tong University, Shanghai, China
PHENG ANN HENG, The Chinese University of Hong Kong, Hong Kong, Hong Kong
MINYI GUO, Shanghai Jiao Tong University, Shanghai, China
CHAO LI, Shanghai Jiao Tong University, Shanghai, China

The emergence of large-scale Mixture of Experts (MoE) models represents a significant advancement in artificial intelligence, offering larger model capacity and computational efficiency through conditional computation. However, deploying and running inference on these models presents significant challenges in computational resources, latency, and energy efficiency. This comprehensive survey analyzes optimization techniques for MoE models across the entire system stack. We first establish a taxonomical framework that categorizes optimization approaches into model-level, system-level, and hardware-level optimizations. At the model level, we examine architectural innovations including efficient expert design, attention mechanisms, various compression techniques such as pruning, quantization, and knowledge distillation, as well as algorithm improvement including dynamic routing strategies and expert merging methods. At the system level, we investigate distributed computing approaches, load balancing mechanisms, and efficient scheduling algorithms that enable scalable deployment. Furthermore, we delve into hardware-specific optimizations and co-design strategies that maximize throughput and energy efficiency. This survey provides both a structured overview of existing solutions and identifies key challenges and promising research directions in MoE inference optimization. To facilitate ongoing updates and the sharing of cutting-edge advances in MoE inference optimization research, we have established a repository accessible at https://github.com/MoE-Inf/awesome-moe-inference/.

Jiacheng Liu and Peng Tang Both authors contributed equally to this research.
This work was supported by the National Natural Science Foundation of China (No. 62441225 and No. U24A20234) and the Research Grants Council of the Hong Kong Special Administrative Region, China, under Project T45-401/22-N..
Authors' Contact Information: Jiacheng Liu, Shanghai Jiao Tong University, Shanghai, Shanghai, China, The Chinese University of Hong Kong, Hong Kong, China, and Hong Kong University of Science and Technology, Hong Kong, China; e-mail: liujiacheng@sjtu.edu.cn; Peng Tang, Shanghai Jiao Tong University, Shanghai, Shanghai, China; e-mail: tttppp@sjtu.edu.cn; Wenfeng Wang, Shanghai Jiao Tong University, Shanghai, Shanghai, China; e-mail: wangwenfeng@sjtu.edu.cn; Yuhang Ren, Shanghai Jiao Tong University, Shanghai, Shanghai, China; e-mail: rtryh125@sjtu.edu.cn; Xiaofeng Hou (corresponding author), Shanghai Jiao Tong University, Shanghai, Shanghai, China; e-mail: hou-xf@cs.sjtu.edu.cn; Pheng Ann Heng, The Chinese University of Hong Kong, Hong Kong, Hong Kong; e-mail: pheng@cse.cuhk.edu.hk; Minyi Guo, Shanghai Jiao Tong University, Shanghai, China; e-mail: guo-my@cs.sjtu.edu.cn; Chao Li (corresponding author), Shanghai Jiao Tong University, Shanghai, China; e-mail: lichao@cs.sjtu.edu.cn.

ACM 0360-0300/2026/03-ART247
https://doi.org/10.1145/3794845

CCS Concepts: • **Computing methodologies** → **Neural networks**; • **Computer systems organization** → **Neural networks**; Embedded and cyber-physical systems;

Additional Key Words and Phrases: Mixture of experts, large language models, inference optimization

ACM Reference Format:
Jiacheng Liu, Peng Tang, Wenfeng Wang, Yuhang Ren, Xiaofeng Hou, Pheng Ann Heng, Minyi Guo, and Chao Li. 2026. A Survey on Inference Optimization Techniques for Mixture of Experts Models. *ACM Comput. Surv.* 58, 10, Article 247 (March 2026), 37 pages. https://doi.org/10.1145/3794845

1 Introduction

Large language models (**LLMs**) have revolutionized artificial intelligence, demonstrating unprecedented capabilities across various domains including natural language processing [21, 131, 187], computer vision [34, 37, 230], and multimodal tasks [94, 141, 193]. Models like GPT-4 [130], Claude [8], and Gemini [179] have achieved remarkable performance in tasks ranging from natural language understanding to complex reasoning and code generation. The impressive capabilities of these models are largely attributed to their massive scale, both in terms of model parameters and computational resources invested in training. This scaling trend is supported by empirical evidence showing consistent improvements in model performance with increased size, as demonstrated by various scaling laws in language modeling and other domains [6, 20, 79]. However, this trajectory presents significant challenges in terms of computational efficiency and resource utilization, particularly during inference, where real-world deployment constraints become critical [10, 204, 222, 239].

Mixture of Experts (**MoE**) has emerged as a promising architectural solution to address scaling challenges in machine learning [160], which was originally introduced by Jacobs et al. in the early 1990s as a method for learning subtasks in neural networks [73]. Numerous MoE-based models [41, 57, 185] have been developed over the years. In the era of LLMs, MoE has again experienced a renaissance [1, 30, 75, 175]. The core principle of MoE is to distribute the model's parameters across multiple specialized sub-networks, or experts, with a learned routing mechanism that selectively activates only the relevant experts for each input. This approach allows models to maintain a large parameter count while keeping computational costs manageable through sparse activation. Recent implementations, such as Mixtral 8x7B [75], DeepSeek-V3 [33], and DBRX [183], have demonstrated the effectiveness of this strategy in scaling language models to trillions of parameters while maintaining reasonable computational requirements.

The success of MoE in scaling models has led to its adoption in various state-of-the-art systems. For example, Google's GLaM [39] outperforms GPT-3 while using significantly fewer computational resources during inference. Similarly, Mixtral 8x7B [75] has demonstrated competitive performance compared to much larger dense models, while maintaining efficient inference characteristics. DeepSeek-V3 [33], a recent open-source MoE model, has surpassed other open-source alternatives and demonstrated performance comparable to prominent closed-source models such as GPT4-o and Claude-3.5-Sonnet. Table 1 summarizes recent state-of-the-art open-source MoE models that have garnered significant attention, showcasing their rapid evolution and widespread adoption across major tech companies and research institutions. The models range in size from 6.7B to 1043B parameters, with varying architectures characterized by different numbers of experts, hidden layers, and attention heads. The chronological progression from 2022 to late 2025 demonstrates increasing model sizes and architectural sophistication, with recent models like DeepSeek-V3 pushing the boundaries in terms of parameter count and model performance. This trend further highlights the strong potential of the MoE architecture in advancing the field of LLMs. These successes have sparked widespread interest in MoE across both academia and industry, leading

Table 1. A List of SoTA MoEs

Reference	Para.	Experts	#L	#H	d_{model}	d_{ffn}	d_{expert}	Affiliation	Time
NLLB [25]	54B	2/64/0	24	16	1024	8192	8192	FaceBook	2022.07
Qwen2-57B-A14B [209]	57.4B	8/64/0	28	28	3584	18944	2560	Alibaba	2023.05
Mixtral-8x7B [75]	46.7B	2/8/0	32	32	4096	14336	14336	Mistral AI	2023.12
OpenMoE [206]	34B	2/16/0	12	12	768	2048	2048	NUS et al.	2023.12
DeepSeekMoE [30]	16.4B	6/64/2	28	16	2048	10944	1408	DeepSeek-AI	2024.01
Qwen1.5-MoE [182]	14.3B	4/60/0	24	16	2048	5632	1408	Alibaba	2024.02
JetMoE [162]	8.52B	2/8/0	24	32	2048	5632	5632	MIT et al.	2024.03
Jamba [104]	51.6B	2/16/0	32	32	4096	14336	14336	ai21labs	2024.03
DBRX [183]	132B	4/16/0	40	48	6144	10752	10752	Databricks	2024.03
Grok-1 [200]	314B	2/8/0	64	48	6144	UNK	UNK	xAI	2024.03
Arctic [150]	482B	2/128/0	35	56	7168	4864	4864	Snowflake	2024.04
Mixtral-8x22B [75]	141B	2/8/0	56	48	6144	16384	16384	Mistral AI	2024.04
DeepSeek-V2 [32]	236B	6/160/2	60	128	5120	12288	1536	DeepSeek-AI	2024.04
Skywork-MoE [197]	13B	2/16/0	52	36	4608	12288	12288	Kunlun Tech	2024.05
Yuan2 [198]	40B	2/32/0	24	16	2048	8192	8192	IEIT-Yuan	2024.05
LLaMA-MoE [241]	6.7B	2/8/0	32	32	4096	11008	11008	Zhu et al.	2024.06
OLMoE [122]	6.92B	8/64/0	16	16	2048	1024	1024	AllenAI	2024.07
Phi-3 [1]	41.9B	2/16/0	32	32	4096	6400	6400	MicroSoft	2024.08
GRIN-MoE [108]	41.9B	2/16/0	32	32	4096	6400	6400	MicroSoft	2024.09
Hunyuan-Large [175]	389B	1/16/1	64	80	6400	18304	18304	Tencent	2024.11
DeepSeek-V3 [33]	671B	8/256/1	61	128	7168	18432	2048	DeepSeek-AI	2024.12
MiniMax-Text-01 [121]	456B	2/32/0	80	64	6144	9216	9216	MiniMax-AI	2025.1
DeepSeek-R1 [31]	671B	8/256/1	61	128	7168	18432	2048	DeepSeek-AI	2025.1
Llama 4 Maverick [119]	402B	1/128/0	48	40	5120	16384	8192	Meta	2025.4
Qwen3-235B-A22B [208]	235B	8/128/0	94	64	4096	12288	1536	Alibaba	2025.5
ERNIE-4.5 [11]	300B	8/64/0	54	64	8192	28672	3584	Baidu	2025.6
Hunyuan-A13B [184]	80B	8/64/1	32	32	4096	24576	3072	Tencent	2025.6
Kimi-K2 [85]	1,043B	8/384/1	61	64	7168	18432	2048	MoonshotAI	2025.7
GPT-oss [3]	120B	4/128/0	36	64	2880	11520	2880	OpenAI	2025.8
GLM-4.5 [224]	355B	8/160/1	92	96	5120	12288	1536	Z.ai	2025.8
LongCat [181]	560B	12/512/0	28	64	6144	12288	2048	Meituan	2025.9

Param. represents the number of total parameters. Experts are listed according to the format of the number of activation experts, total experts, and shared experts. #L represents the number of hidden layers, #H represents the number of attention heads. d_{model} is the hidden size, d_{ffn} is the intermediate size of FFNs, d_{expert} is the intermediate size of FFN experts.

to innovations in model design [23, 195, 228], training techniques [38, 51, 110], and deployment strategies [16, 17, 217].

However, the efficient deployment of MoE models for inference presents unique and significant challenges [70, 177, 215, 234]. The dynamic nature of expert activation patterns (the computation path is determined by the input, see Section 2 for details) introduces complexity in resource management and scheduling, that is, not present in traditional dense models. At the model level, the design of efficient expert architectures faces challenges in balancing model capacity with computational efficiency, while routing mechanisms must optimize expert selection and load distribution. The system-level challenges are particularly complex: distributed computation requires sophisticated scheduling algorithms to manage expert placement and activation, load balancing must handle dynamic workload variations across experts, and memory management needs to efficiently handle the loading and unloading of expert parameters. Hardware-level challenges stem from the fundamental mismatch between traditional hardware architectures optimized for dense computation and

the sparse, dynamic nature of MoE inference. This necessitates specialized acceleration techniques to handle sparse computation patterns, manage memory bandwidth efficiently, and provide flexible computation capabilities for dynamic expert switching. Communication overhead in distributed settings presents another significant challenge, particularly when experts are distributed across different devices or nodes, requiring careful optimization of data movement and synchronization.

Numerous methods have been developed to address these challenges in MoE deployment and inference [77, 144, 156, 202]. While the rapid growth of research in this field demonstrates its importance, it can also make it difficult to identify key trends and best practices. A critical gap in the existing literature is the absence of a systematic framework for analyzing and developing comprehensive inference optimization solutions for MoE models. To bridge this gap, our work offers a comprehensive survey of inference optimization techniques for MoE models. We propose a taxonomical framework that categorizes optimization approaches into model-level, system-level, and hardware-level optimizations, as illustrated in Figure 1. This framework provides a structured approach to understanding and comparing different optimization techniques. While there are related surveys on LLM efficiency [10, 92, 98, 186, 189, 204, 222, 239] and MoE architectures [14, 45, 188], our work is the first to specifically focus on inference optimization techniques for MoE models. We systematically analyze optimization approaches at different abstraction levels, from model architecture to hardware acceleration, providing a valuable resource for researchers and practitioners working deploy MoE models for different real-world applications.

The remainder of this survey is organized as follows: Section 2 provides background on MoE models and their inference characteristics. Sections 3–5 detail optimization techniques at the model, system, and hardware levels, respectively. Section 6 discusses future challenges and opportunities, and Section 7 concludes the survey.

2 Fundamentals of Mixture of Experts

MoE represents a significant architectural paradigm in neural networks, particularly in LLMs, where it enables conditional computation through sparse activation mechanisms [14]. At its core, each layer of an MoE architecture consists of a routing network $R(x)$ and a set of expert networks $E_1, E_2, ..., E_N$, where N denotes the total number of experts. The fundamental principle of MoE block can be expressed as $y = \sum_{i=1}^{N} g_i(x) \cdot E_i(x)$, where $g_i(x)$ represents the gating function for expert i, and $E_i(x)$ is the output of expert i.

As illustrated in Figure 2, existing studies typically use the MoE module to replace part of the traditional dense layer, thereby forming a sparse MoE layer. While most research focuses on substituting the FFN module with the MoE module [1, 32, 75, 76, 182], some have also explored replacing the attention module [77, 162, 163, 228].

The typical MoE model inference procedure follows a sequence of expert selection, parallel computation, and output aggregation. First, the router computes expert selection probabilities:

$$\theta = \text{Softmax}(R(x)), \tag{1}$$

where $x \in \mathbb{R}^d$ is the input token embedding, $R(\cdot)$ is the routing function, and $\theta \in \mathbb{R}^N$ represents the expert selection probabilities for N total experts. Next, the top-k experts are selected based on these probabilities:

$$E_{\text{selected}} = \text{TopK}(\theta, K), \tag{2}$$

where E_{selected} contains the indices of the K selected experts, $K \leq N$. The selected experts then process the input in parallel:

$$y_i = E_i(x), \quad \forall i \in E_{\text{selected}}, \tag{3}$$

where $E_i(\cdot)$ represents the computation of expert i, and y_i is its output.

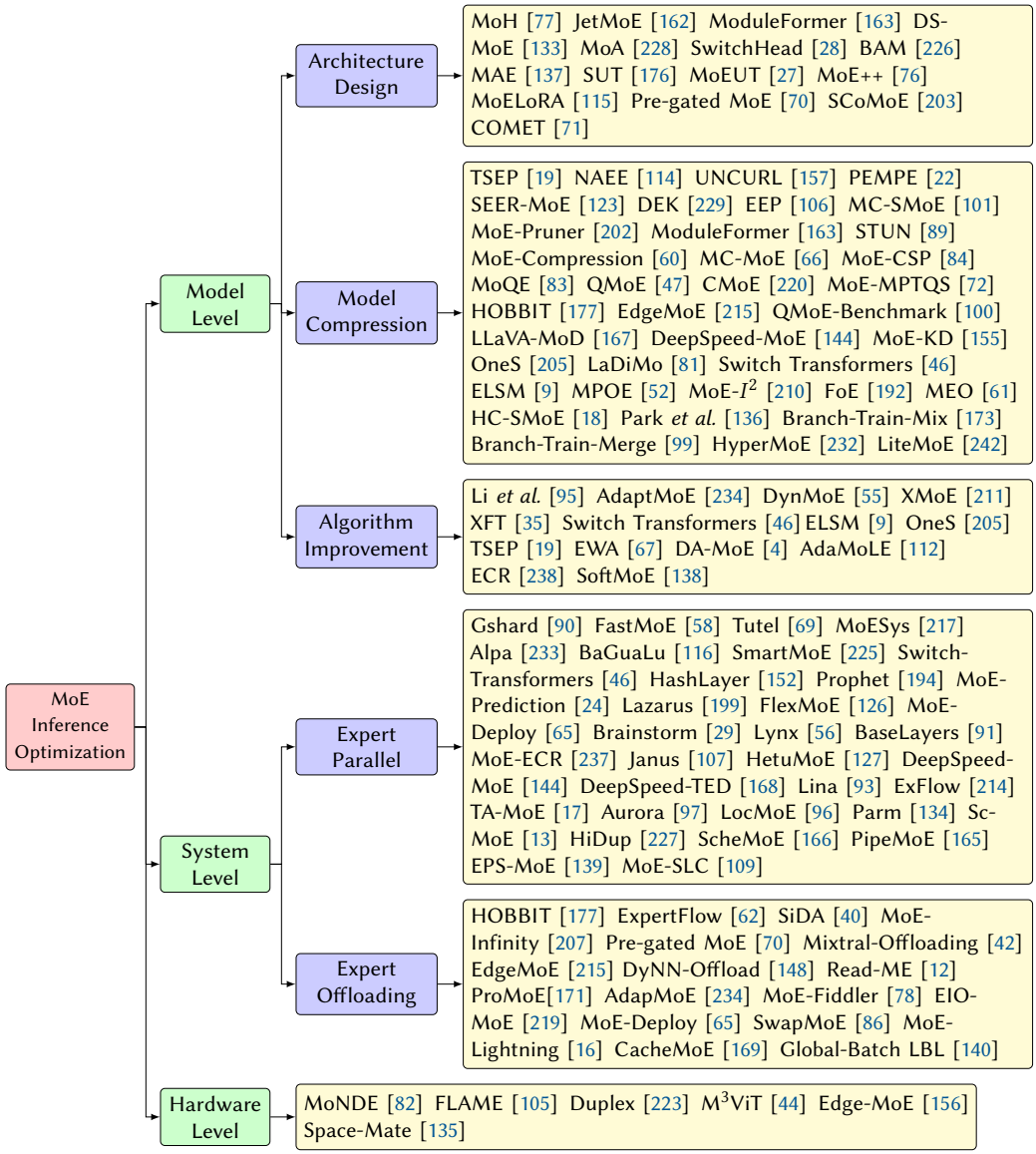


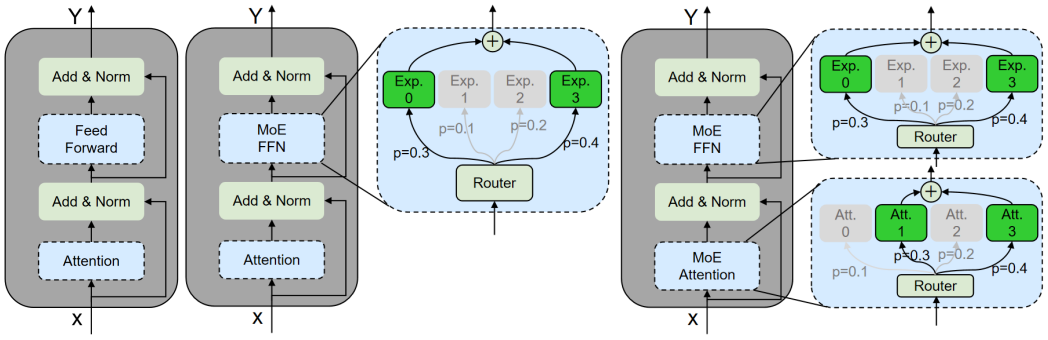
Fig. 1. Taxonomy of MoE inference optimization.

Finally, the expert outputs are combined through a weighted aggregation. In many MoE implementations, the routing weights, θ_i, are normalized to form a probability distribution over the selected experts. This is illustrated in Equation (4):

$$y = \sum_{i \in E_{\text{selected}}} \frac{\theta_i}{\sum_{j \in E_{\text{selected}}} \theta_j} \cdot y_i, \tag{4}$$

where the weights are normalized by their sum.

However, it is important to note that not all MoE approaches normalize these weights. For example, Switch-Transformer [46] uses a softmax to determine expert choice without afterwards



(a) Dense Layer (b) MoE FFN Layer (c) MoE Attention+FFN Layer

Fig. 2. Architectural comparison of dense layer with MoE layers: (a) conventional dense transformer layer, (b) transformer layer with MoE-based feed-forward network, and (c) transformer layer with both MoE-based attention and feed-forward networks.

normalization, simplifying the weighting scheme. In this case, the expert outputs is:

$$y = \sum_{i \in E_{\text{selected}}} \theta_i \cdot y_i. \tag{5}$$

More explicitly, some models provide a choice in this regard. The Qwen2-MoE [209] model, for example, includes a parameter named "norm_topk_prob" that specifically controls whether the routing weights for the top-k selected experts are normalized. This highlights that the normalization of expert weights is a key design choice in MoE architectures, with different models adopting different strategies.

The MoE architecture offers several compelling advantages that have contributed to its growing adoption in modern neural networks. First, conditional computation yields substantial efficiency gains. By activating only a subset of task-relevant experts for each input token, MoE models reduce computational cost while sustaining performance comparable to dense models with equivalent capacity. This property is particularly valuable in resource-constrained settings and when scaling to larger model sizes.

Second, expert specialization enhances modeling fidelity. Individual experts acquire domain- or pattern-specific competence, enabling more precise and targeted computations. This effect is especially beneficial in language modeling, where different experts can focus on distinct linguistic phenomena, domains, or tasks.

Third, the dynamic routing mechanism enables adaptive computation based on input complexity. More challenging or nuanced inputs can engage multiple experts with complementary specializations, while simpler inputs might require only a small subset of experts. This adaptive resource allocation helps optimize the computation-performance trade-off and potentially improves both efficiency and effectiveness.

By partitioning dense models into relatively independent expert models and dynamically activating specific subsets (or the entire set) of experts based on each input token, the model's overall performance can be significantly enhanced with only a marginal increase in inference computation. This approach clearly demonstrates the MoE model's exceptional flexibility and scalability.

3 Model-level Optimizations

Model-level optimizations aim to enhance the inherent structure and efficiency of MoE models through systematic improvements in architecture, parameter optimization, and algorithmic design.

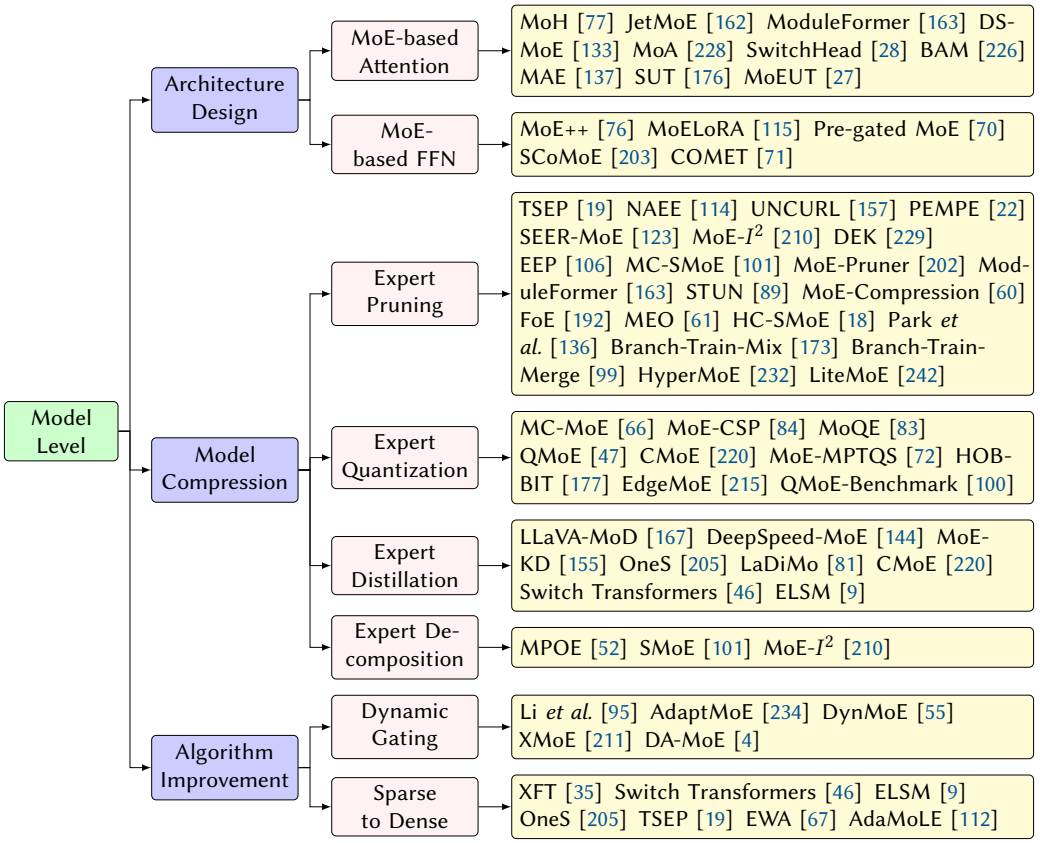


Fig. 3. Model-level inference optimization techniques for MoE.

These optimizations can be broadly categorized into three main areas: efficient model architecture design, model compression techniques, and algorithmic improvements. Architecture design focuses on developing more efficient expert and attention structures, while compression techniques aim to reduce model size and memory footprint through methods such as pruning, quantization, and knowledge distillation. Algorithmic improvements concentrate on enhancing the dynamic aspects of MoE models, including routing mechanisms and expert combination strategies. Figure 3 illustrates the detailed taxonomy of model-level optimization, that is, described in this section.

3.1 Efficient Model Architecture Design

A transformer block typically consists of two main components: the attention module and the FFN module. To build better MoE models, many studies focus on designing improved versions of both the attention and FFN modules, aiming to achieve strong performance while maintaining high efficiency.

3.1.1 MoE-based Attention Design. In addition to the typical application of the MoE structure in the FFN module of the transformer layer, current studies explore how to incorporate MoE into the attention module for improved performance. MAE [137] was the first to interpret multi-head attention as an MoE system, using a learned gating function to activate different experts—each composed of $n-1$ heads, where n denotes the number of attention heads per layer. However, its fixed expert composition limits flexibility in head selection. To address this, MoA [228] and BAM [226]

enable dynamic selection of k individual heads per token and share key/value projections across heads, significantly reducing memory overhead while preserving expressivity. SwitchHead [28] further improves efficiency by sharing query and key projections, though at the cost of reduced routing diversity. MoH [77] introduces shared heads and a two-stage routing mechanism, achieving better performance than MoA by balancing specialization and redundancy. Building on these ideas, ModuleFormer [163] extends sparsity to both attention and FFN layers, enabling modular model scaling. This design is adopted in JetMoE-8B [162], a high-performing open-source model with fully sparse layers. In contrast, DS-MoE [133] proposes a hybrid dense-training/sparse-inference paradigm, which eases optimization during training while retaining inference efficiency. Meanwhile, SUT [176] and MoEUT [27] achieve extreme parameter sharing across layers via sparse universal transformers, trading off some capacity for compactness and faster convergence.

3.1.2 MoE-based FFN Design. To enhance the efficiency of MoE-based models, current research explores various variants of the standard MoE module. MoE++ [76] introduces zero-computation experts that skip actual computation for certain tokens, effectively reducing FLOPs without degrading performance—particularly useful in latency-sensitive scenarios. SCoMoE [203] tackles the communication bottleneck in distributed MoE by adopting a hierarchical all-to-all communication pattern, which scales better on large clusters but requires careful topology-aware scheduling. Pregated MoE [70] improves inference latency on memory-constrained devices by prefetching experts based on early gating signals, though it assumes predictable routing patterns. COMET [71] replaces the standard linear router with a tree-based selector, enabling more fine-grained expert assignment at the cost of increased routing complexity. Additionally, MoELoRA [115] reimagines LoRA as a lightweight MoE, enabling parameter-efficient fine-tuning via routed adapters and offering a compelling trade-off between adaptability and storage overhead. MoE-Prism [201] offers a novel method to deconstruct monolithic experts into fine-grained sub-experts, which can enable more flexible and efficient MoE service delivery.

3.2 Model Compression Techniques

Model compression is a popular area of research for reducing model size in current LLM studies, with techniques such as pruning [102, 118], quantization [48, 191], knowledge distillation [2, 54], and low-rank decomposition [196, 221]. Since experts constitute the majority of the weights in MoE models (e.g., 96% for Mixtral-8x7B [75]), most MoE-related compression efforts focus on applying these common techniques specifically to the experts. Notably, pruning and quantization in MoE architectures must preserve expert diversity; overly aggressive compression can collapse expert specialization and degrade performance, a challenge less pronounced in dense models.

3.2.1 Expert Pruning. Due to the large number of parameters in MoE-based models, current research explores pruning methods to reduce the number of parameters in MoE experts. These methods are generally divided into structured and unstructured pruning. Most studies focus on structured expert pruning, which aims to reduce the number of experts in the MoE layer while maintaining model accuracy. As shown in Figure 4(a), some approaches directly remove unimportant experts (left side), while others merge groups of experts into a single expert (right side). Representative methods are summarized in Table 2. TSEP [19] removes non-professional experts for the target downstream task while fine-tuning professional experts to preserve model performance. They also demonstrate the superiority of the eager expert pruning paradigm over other possible solutions like two-pass optimization or staged expert pruning. NAEE [114] eliminates unimportant experts by evaluating expert combinations on a small calibration dataset to minimize accuracy loss, simultaneously reducing model sizes and increase inference speed, while UNCURL [157] reduces the number of experts based on MoE router logits. Some insights useful for model design choices

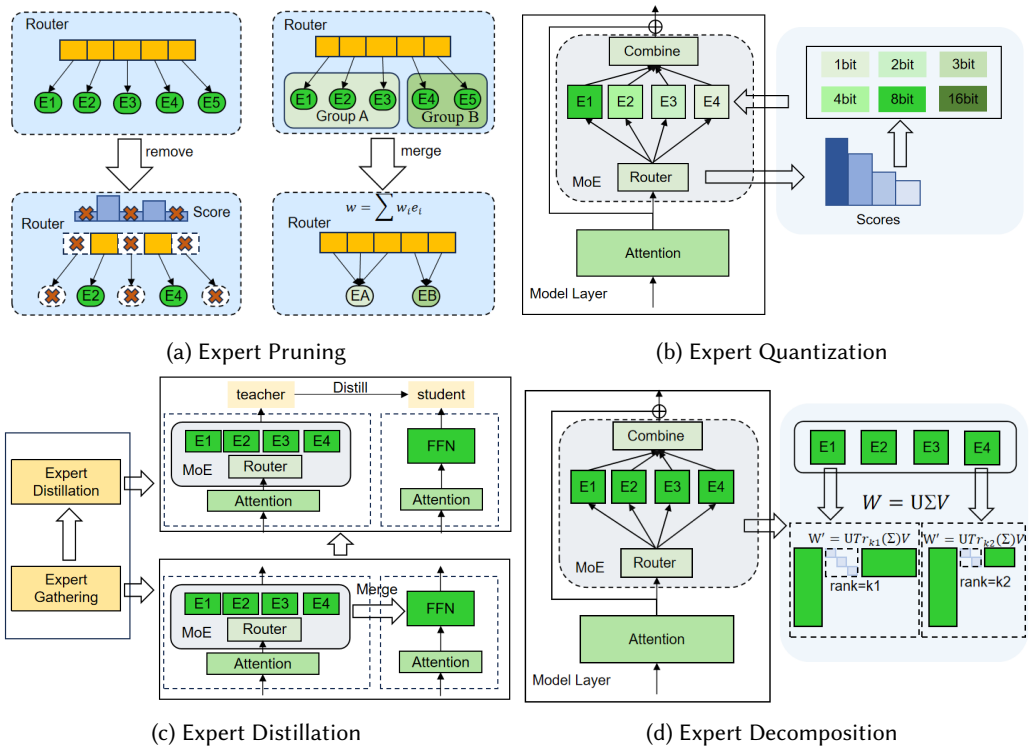


Fig. 4. Compression techniques for MoE models.

Table 2. A comparison of Pruning Methods

	Sparsity	TSA	Datasets of Fine-tuning	Structured		Unstructured
				Delete	Merge	
TSEP [19]	96.875%	S	GLUE [190] SQuAD [146]	✓		
NAEE [114]	50%	S	MetaMathQA [218]	✓		
UNCURL [157]	75%	S	FLAN [113]	✓		
PEMPE [22]	75%	A	CIFAR-10 CIFAR-100 [87] ImageNet [153]	✓		
SEER-MoE [123]	25%	S	MMLU [63] SST5 [170]	✓		
MoE-I^2 [210]	53.98%	A	Alpaca [178]	✓		
DEK [229]	75%	S	C4 [143]		✓	
EEP [106]	75%	A	NA	✓	✓	
MC-SMoE [101]	75%	S	eight datasets		✓	
MoE-Pruner [202]	50%	A	C4 [143]			✓
STUN [89]	70%	A	C4 [143]			✓
MoE-compression [60]	50%	A	Lima [235] MetaMathQA [218]	✓		✓

Sparsity indicates the max ratio of pruning. TSA represents task specific or agnostic. EEP [106] does not require additional fine-tuning. Finetuning datasets of MS-SMoE [101] are SST2 [170], MRPC [36], MultiRC [80], COPA [151], WinoGrande [154], SQuAD [146], WikiQA [212], and HotpotQA [213].

considering task-specific inference optimization can give guidance for later stages. PEMPE [22] prunes experts that have smaller changes in the router's l_2 norm between the pre-trained and fine-tuned models, and will conduct experiments with other compression techniques on SOTA vision MoEs. SEER-MoE [123] employs a heavy-hitters counting method for expert pruning, then with regularization-based fine-tuning reaches further expert pruning. MoE-I^2 [210] proposes the

Layer-wise Genetic Search and Block-wise KT-Reception Field with the non-uniform pruning ratio to prune unimportant experts.

In addition to direct pruning, some studies utilize expert merging to reduce the number of experts. For unstructured pruning, MoE-Pruner [202] prunes weights with the smallest magnitudes, weighted by the corresponding input activations and router weights. The pruned MoE models can benefit from a pre-trained teacher model through expert-wise knowledge distillation and have compatibility with structured pruning. Moreover, STUN [89] combines structured and unstructured pruning to achieve better performance than unstructured pruning alone, utilizing the sparse character of MoEs based on behavior similarity in a greedy manner. MoE-Compression [60] proposes a unified framework for MoE compression, using both structured and unstructured pruning methods to achieve significant inference speedup with minimal accuracy loss.

Another potential approach is merging outdated experts based on their parameters to achieve better performance. Branch-Train-Merge [99] independently trains different parts of the model on distinct subsets of data, eliminating the need for large-scale multi-node synchronization typically required in traditional LLM training. Building on this, Branch-Train-Mix [173] trains multiple copies of a seed LLM to specialize in multiple domains in an asynchronous and parallel fashion, then merges the parameters of the MoE layer to create a unified model that can be further trained. The second finetuning stage makes the final LLM more performant. They also find that their approach is more computing efficient compared to the dense model or specialized MoE model on training. Park et al. [136] observed that introducing a shared layer in the MoE could lead to performance degradation. To address this, they trained merged experts that learned the same features in different ways, improving their ability to generalize and mitigating catastrophic forgetting during incremental learning of multi-domain tasks. MC-SMoE [101] divides experts into different groups based on routing policies and then merges each group into one expert. HC-SMoE [18] is a framework that uses hierarchical clustering to merge experts without requiring retraining and can be applied in a task-agnostic manner. During inference, MEO [61] systematically investigates the computational cost of MoE. In the drop-in replacement algorithm, they first merge the selected expert parameters and then perform efficient computation. FoE [192] fuses the outputs of expert models by leveraging their complementary knowledge of the data distribution, framing it as a supervised learning instance. DEK [229] identifies and groups similar experts in feature space, then merges experts within the same group in weight space to reduce the expert count. EEP [106] introduces a two-stage approach to both prune and merge experts, reducing the total number of experts (saving GPU memory) and the number of active experts (accelerating inference). Inspired by the concept of knowledge transfer in multi-task learning, HyperMoE [232] proposes a novel expert network that further increases sparsity while utilizing information from unselected experts as supplementary input. In practice, they capture the contextual information of experts to compensate for the performance loss of transferring knowledge to specific experts. LiteMoE [242] retains the most critical experts based on the application's characteristics, merges secondary experts, and obtains the final sparse model without retraining. This approach enables efficient deployment of lightweight submodels on resource-constrained mobile devices.

3.2.2 Expert Quantization. In addition to model pruning, quantization is an effective technique for reducing model size by converting high-precision weights into low-precision. Given the redundancy of experts in MoE models, current research primarily focuses on quantizing the weights of the experts in MoE. As illustrated in Figure 4(b), experts are quantized into appropriate low-precision versions using various methods. MC-MoE [66] leverages the access frequency and activation weight to assess the importance of each expert. Along with the associated quantization loss, these metrics are used to determine the optimal quantization configuration for experts via the Integer

Table 3. A Comparison of Quantization Methods

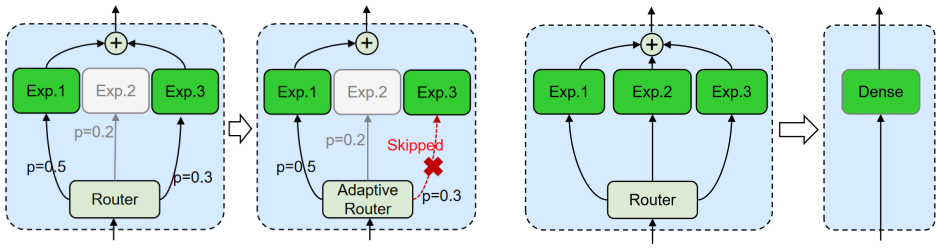
Method	Quantization Type		Memory Reduction	Accuracy Drop	Inference Speedup	Quantization Bits
	Weight	Activation				
MC-MoE [101]	✓		4.27x	3.8%	1.80x	1, 2, and 3
MoE-CSP [84]	✓		4.00x	-	26.00x	4, 8
MoQE [83]	✓		4.90x	0.97%	-	2, 3, and 4
QMoE [47]	✓		20x	6.7%	0.95x	1, 2
CMoE [220]	✓	✓	150x	23.81%	-	1, 2, and 4
MoE-MPTQS [72]	✓		-	0 ∼ 4.98%	1.00x ∼ 20.63x	4, 8
HOBBIT [177]	✓		-	1%	1.35x	2, 4
EdgeMoE [215]	✓		1.05x ∼ 1.18x	5%	1.11x ∼ 2.78x	2, 4, and 8

Programming model. Specifically, the ith expert's access frequency is defined as $\phi_i = \frac{n_i}{N}$, and the activation weight is defined as $w_i = \frac{\sum_{j=1}^{N} \sigma_j}{N}$, where n_i is the usage frequency, σ_j is the routing weight, and N represents the size of the calibration dataset. The quantization loss, ϵ_{ij} (computed using the Frobenius norm), is then determined for quantizing expert i to j bits ($j \in 1, 2, 3$). Using these metrics, MC-MoE defines the objective function as $\sum_{i=1}^{n} \sum_{j=1}^{3} \phi_i^{\alpha} \cdot w_i^{\beta} \cdot (\epsilon_{ij} \cdot x_{ij})^{\gamma}$, which is minimized using Integer Programming to determine the optimal bit-width for each expert. MoE-CSP [84] quantizes expert weights to either 4 or 8 bits to reduce memory consumption in MoE models. Additionally, it designs specific CUDA kernels that handle the 4-bit/8-bit quantized weights and perform floating-point calculations to accelerate computations. MoQE [83] quantizes expert weights to 2 bits to address the memory and latency challenges of MoE models based on its observations that quantizing expert FFN layers to 2 bits does not significantly affect model quality, while quantizing other components, like self-attention, significantly hurts performance. Further advancements include QMoE [47] and CMoE [220], which aggressively compress MoE model into just 1-bit. QMoE implements a highly scalable compression algorithm for large models and introduces a custom compression format along with bespoke GPU kernels for efficient on-the-fly computation. On the other hand, CMoE uses binary-weight networks to quantize model weights to 1-bit and applies learned step-size quantization to activations, quantizing them to 4 bits for MoE-based ASR models, enabling deployment on embedded devices. Moreover, some MoE-optimized systems leverage quantization for better system efficiency. MoE-MPTQS [72] and HOBBIT [177] dynamically select quantized experts to replace original experts based on the current inputs, thereby reducing the expert loading cost on memory-limited devices. EdgeMoE [215] statistically determines the appropriate expert bit-width by profiling expert importance on a calibration dataset. Furthermore, QMoE-Benchmark [100] provides a benchmark for exploring various MoE structure-aware quantization heuristics, from coarse to fine granularity. The study reveals that different MoE structures (e.g., blocks, experts, and linear layers) require different numbers of weight bits for effective and efficient quantization.

We summarize the main results reported by the methods discussed above in Table 3. From the table, we observe that quantization primarily benefits memory reduction, with most methods achieving more than a 4x reduction in memory usage. Some methods also lead to actual inference speedup, while others do not. For instance, QMoE even incurs a 5% overhead due to the absence of a dedicated 1-bit inference CUDA kernel implementation and hardware support. Furthermore, quantization typically causes some accuracy loss, with lower bit widths resulting in greater accuracy degradation. Therefore, when applying quantization to a model, it is crucial to strike a balance between memory consumption, accuracy, and inference speedup, taking into account the specific requirements and available hardware resources.

3.2.3 Expert Distillation. Knowledge distillation is another effective method for creating smaller, yet powerful models from larger ones. As shown in Figure 4(c), knowledge distillation presents a promising solution to generate a compact, high-performance model from the original MoE model. LLaVA-MoD [167] combines the MoE structure with knowledge distillation to efficiently train **small multimodal large language models** (**s-MLLMs**) from large ones (l-MLLMs). It first incorporates the MoE structure into the s-MLLM to balance computational efficiency and model performance. Then, it introduces two consecutive distillation stages, mimic distillation and preference distillation, to train the s-MLLM. Mimic distillation minimizes the **Kullback-Leibler** (**KL**) divergence between the output distributions of the s-MLLM and l-MLLM, enabling the s-MLLM to emulate the l-MLLM's understanding. Preference distillation further refines the s-MLLM using Preference Optimization with additional datasets. DeepSpeed-MoE [144] employs staged knowledge distillation to create a distilled version of its proposed PR-MoE model, called MoS. This method reduces model size while maintaining performance. Additionally, some studies focus on transferring the power of sparse models to dense models through knowledge distillation for more efficient deployment. For example, OneS [205] generates a dense model from a MoE model in two steps: knowledge gathering and knowledge distillation. Knowledge gathering merges experts into a single expert using four aggregation methods, including summation, averaging, top-k knowledge gathering, and **Singular Value Decomposition** (**SVD**) knowledge gathering. In the second step, knowledge distillation distills the merged model using the original MoE model. What's more, MoE-KD [155] and CMoE [220] distill MoE-based speech recognition models into dense models, accelerating the speech recognition process. Specifically, MoE-KD initializes the FFN module of the student dense model with the most frequently used expert from the teacher MoE model, then trains the FFN modules of the student model through layer-wise distillation. CMoE distills a binary dense model from the original model using quantization techniques. Switch Transformers [46] and ELSM [9] also explore distillation techniques to convert their sparse models into dense models. Additionally, to simplify the construction of MoE models, LaDiMo [81] converts a dense model into a sparse MoE model via layer-wise distillation, where each expert learns to approximate the results of the original dense layers.

3.2.4 Expert Decomposition. As shown in Figure 4(d), low-rank decomposition is an effective method for reducing model size by decomposing a large weight matrix into smaller matrices. MPOE [52] employs the **matrix product operator** (**MPO**), a tensor decomposition technique derived from quantum many-body physics, to decompose the expert weight matrix into a central tensor and several auxiliary tensors. The central tensor retains most of the parameters and core information of the original weight matrix, while the auxiliary tensors are much smaller and serve as complements to the central tensor. After decomposition, all experts in a layer share the same central tensor, thereby significantly reducing the total number of parameters in that layer. MC-SMoE [101] first groups the experts into several clusters and then merges each group into a single expert using a weighted sum. Low-rank decomposition is then applied to the merged experts to further reduce the model size. This approach is based on the observation that the merged experts have a lower rank compared to the original experts, making them more suitable for decomposition. MoE-I^2 [210] identifies the importance of each expert and assigns higher ranks to more important experts while assigning lower ranks to less important ones for low-rank decomposition. The importance, $I_{i,j}$, of the jth expert in the ith layer, $e_{i,j}$, is determined by removing $e_{i,j}$ and calculating the performance loss compared to the original model. The SVD rank, $r_{i,j}$, for $e_{i,j}$ is then computed as $r_{i,j} = \left\lfloor \frac{(I_{i,j}+\epsilon)^\alpha}{\sum_{j=1}^{M_i}(I_{i,j}+\epsilon)^\alpha} \right\rfloor \cdot R_a \cdot M_i$, where ϵ and α are hyperparameters, R_a is the compression ratio, and M_i is the number of experts in layer i.



(a) Experts Skipping (b) Sparse to Dense

Fig. 5. Algorithm improvement on expert layers.

3.3 Algorithm Improvement

In this part, we conduct an in-depth exploration of two other different strategies aimed at improving the MoE model inference algorithm.

3.3.1 Dynamic Gating. Given the significant progress made in utilizing the sparsity of MoE models, specific strategies can be employed to further exploit this sparsity in the inference process. Due to the vertical parallel structure of MoE experts, dynamically reducing the number of experts activated for each token clearly presents an effective strategy. Figure 5(a) illustrates the calculation process of MoE layer when the skip mechanism is applied, in contrast to the original process.

Li et al. [95] proposed a self-adaptive gating mechanism that dynamically determines the number of experts required for each token, based on the expert probability distribution. This method enhances training efficiency while preserving the sparsity of the MoE model and further reduces training time through curriculum learning. DynMoE [55] eliminates the need for fixed top-k hyperparameters by enabling each token to dynamically select how many experts to activate based on its complexity. It jointly optimizes a gating network that predicts both the number and identity of experts per token, reducing computational overhead while maintaining model performance. XMoE [211] employs a strategy of using smaller, but more experts, with dynamic expert activation based on threshold values to balance computational efficiency and model performance. Adap-MoE [234] dynamically adjusts the number of activated experts per layer based on its sensitivity to expert sparsification—reducing experts in insensitive layers while preserving them in accuracy-critical ones. This layer-wise adaptive gating cuts average expert usage by 25% without accuracy loss, and is complemented by expert prefetching and cache management to lower inference latency on edge devices. The comprehensive comparison of performance and methods of these works is presented in Table 4. DA-MoE [4] introduces a token importance prediction method derived from attention mechanisms to guide expert allocation. By assigning experts based on token importance scores, their approach achieves strong performance on the GLUE benchmark while demonstrating effective scaling with increased expert count.

3.3.2 Sparse to Dense. In certain scenarios, dense models offer unique advantages due to their smaller number of parameters. Therefore, transforming MoE models into dense target models all at once can achieve maximum sparsity while maintaining model performance. Most approaches use knowledge distillation to achieve this sparse-to-dense conversion, as illustrated in Figure 5(b).

XFT [35] proposes a novel method to supervise fine-tune dense LLMs. It first generate a sparse-upcycled MoE model, and then transform it back into an efficient dense LLM of the same size and structure through a learnable merging mechanism. Switch Transformers [46] explores distillation techniques to convert a sparse model into a dense one, demonstrating that the dense model can retain over 30% of its performance even after compressing 97% of the parameters from

Table 4. A Comparison of Dynamic Gating Methods

Method	FLOPs Reduction	Speedup	Threshold Strategy	Load Balance	PR.
Fixed top-k gating	0%	1.0x	✗	✗	✓
Li et al. [95]	38.2%	1.32x	accumulative probability	soft on top-1	✓
DynMoE [55]	9%	1.37x	single expert probability	✓	✗
XMoE [211]	75%	-	accumulative probability	✓	✓
AdapMoE [234]	25% of experts	1.35x	performance perturbation	✗	✓

PR. indicates performance Retention. Specially, Li et al. [95] only uses soft load balance constraints on top-1 gating. No specific acceleration ratio was provided in XMoE [211].

the sparse MoE model. ELSM [9] demonstrates that dense student models distilled from sparse MoE teachers can match and even surpass the teacher's performance. OneS [205] employs four distinct methods to generate the dense model, including summation, averaging, top-k Knowledge Gathering, and SVD Knowledge Gathering. The model is then refined to reduce noise by gathering sparse knowledge. TSEP [19] progressively eliminates non-expert components based on specific downstream tasks, ultimately converting the sparse MoE model into a dense counterpart. EWA [67] replaces FFNs with specially designed MoEs during training before reverting to dense ViT for inference. AdaMoLE [112], which combines the **Low-Rank Adaptation** (**LoRA**) structure, a dedicated network is used to adjust the activation threshold for different task complexities. It has shown superior performance to baseline in many natural language processing tasks, especially in some commonsense reasoning tasks. **expert-choice routing** (**ECR**) [238] exploits the relative importance of different tokens, so each expert in MoE block can have a fixed bucket size to avoid an expert being under-specialized or over-specialized. SoftMoE [138] introduces a fully differentiable sparse MoE architecture that replaces hard token-to-expert routing with implicit soft assignment. This design outperforms both dense Transformers and hard-routing MoEs in visual recognition tasks.

4 System-level Optimizations

Due to the unique structure of MoE, many studies focus on accelerating inference at the system level by leveraging the sparse activation patterns inherent to this architecture. Typically, MoE models are deployed in two scenarios: cloud environments with multiple powerful servers and edge environments with a single device. In cloud clusters, MoE models are distributed across many devices to enable parallel execution. In addition to traditional parallelization techniques such as data parallelism, tensor parallelism, and pipeline parallelism [74, 124, 145], expert parallelism is a specialized method tailored specifically for MoE models. On edge devices, limited GPU memory often cannot accommodate all parameters of an MoE model, necessitating the offloading of some parameters to CPU memory or SSD storage. To address this, the expert-offloading technique has been developed to fully utilize the sparse activation patterns of experts for efficient execution. Figure 6 shows the detailed structure of this section.

4.1 Expert Parallelism

Expert parallelism is a key technique for deploying large MoE models across multiple devices to enable distributed execution. As Figure 7(a) shows, when distributing MoE layers, each device holds a subset of experts along with all other non-expert parameters [166]. In special cases where an expert is very large, each device may contain only one expert. During the execution of an MoE layer, inputs are processed on each device by the attention module and gate module. Subsequently, an all-to-all communication operation redistributes tokens to corresponding devices based on the gate output, allowing each expert to process its assigned inputs. Finally, another all-to-all communication operation occurs to exchange the experts' outputs and send them back to the

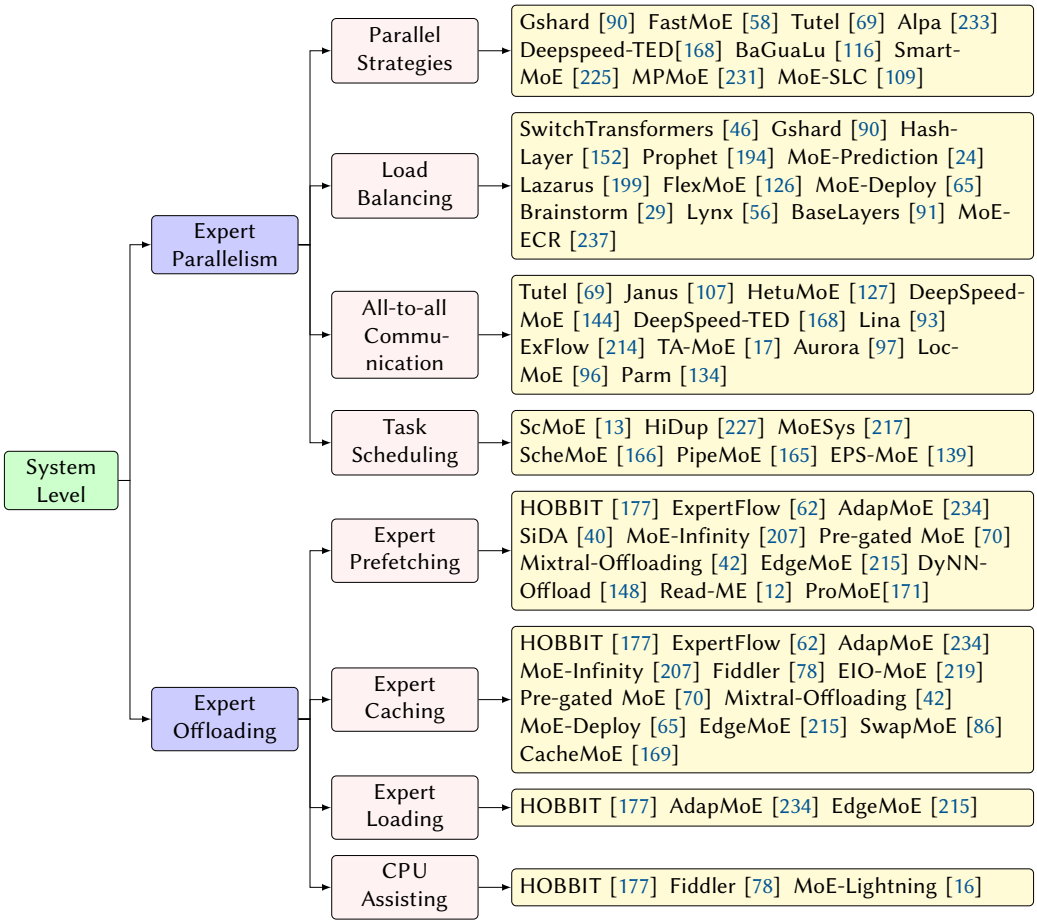
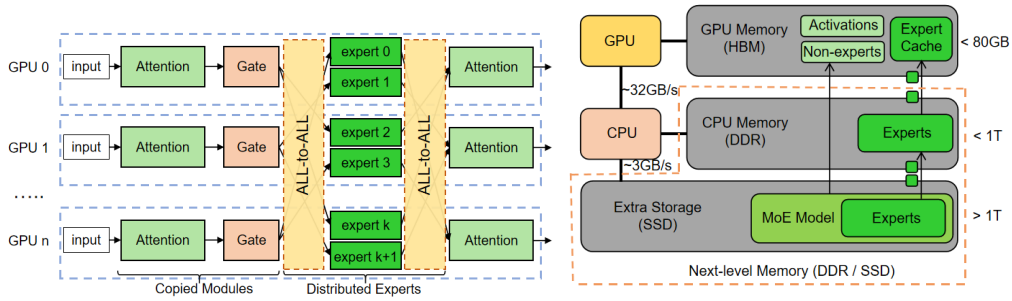


Fig. 6. System-level inference optimization techniques for MoE.

originating devices. Consequently, the execution time of MoE layers is dominated by two primary phases: computation and communication. To accelerate inference, various studies have proposed optimizations for these phases.

Existing research focuses on optimizing MoE training and inference performance along four main dimensions: parallelism strategy, load balancing, all-to-all communication, and task scheduling.

4.1.1 Parallelism Strategy Design. In addition to relying solely on expert parallelism [58, 90], recent studies combine multiple parallel strategies to maximize parallelism and achieve more efficient distributed computing. Tutel [69] dynamically switches parallelism strategies at each iteration without incurring extra overhead. Specifically, this work employs a single distribution layout that encompasses all optimal strategies, thereby eliminating the need to reformat input data or weights when switching parallelism strategies. Alpa [233] reclassifies conventional data and model parallelism into intra-operator and inter-operator parallelism based on the key observation that different parallelization techniques have varying bandwidth requirements for communication. In typical clusters, co-located devices have high communication bandwidth, whereas distant devices have limited communication bandwidth. With this reclassification, Alpa can automatically derive efficient parallel execution plans that incorporate data, operator, and pipeline parallelism based



(a) Expert Parallelism (b) Expert Offloading

Fig. 7. Expert parallelism and expert offloading techniques.

on the model description and cluster configuration. DeepSpeed-TED [168] implements a hybrid parallel algorithm combining data, tensor, and expert parallelism. It designs a tiled version of the optimizer to reduce peak memory consumption arising from the combination of these three parallelism forms, thereby enabling the training of MoE models with 4–8× larger base models than the current state-of-the-art. BaGuaLu [116] proposes a parallel strategy called MoDa that combines MoE parallelism and data parallelism to effectively scale models. Specifically, MoDa places data parallelism within a single supernode while maintaining MoE communication globally across supernodes. This approach is based on the observation that All-reduce communication (in data parallelism) incurs much higher costs than All-to-all communication (in MoE parallelism) during the training of its target large-scale models. SmartMoE [225] explores the space of hybrid parallelism with an awareness of heterogeneous workloads, incorporating potentially faster candidate parallel strategies. It introduces a two-stage approach to identify optimal parallel strategies for data-sensitive models, periodically determining whether a new parallel strategy should be employed at runtime. MPMoE [231] proposes a lightweight profile-based algorithm that leverages profiling information to identify the most suitable configuration at runtime, optimizing pipeline parallelism. Additionally, in serverless computing, MoE-SLC [109] proposes a Bayesian optimization framework with multi-dimensional δ-greedy search to learn expert selections for expert parallelism and optimize the MoE deployment to achieve optimal billed cost. By enforcing load balancing at the global-batch (corpus) level rather than per-sequence, global-batch LBL [140] enables experts to specialize by domain during training. At inference time, this leads to more consistent and accurate routing decisions, improving both model quality and computational efficiency without increasing FLOPs.

4.1.2 Load Balancing. Token processing in MoE models can lead to imbalances, where some experts receive significantly more tokens than others. This disparity results in certain devices experiencing high overheads while others remain idle, ultimately delaying overall computation. To address this issue, some works propose adding auxiliary load balancing loss terms to train the gate module [46, 90, 160], while others implement gate modules with special hash functions to prevent load imbalance [152]. Advanced techniques focus on optimizing expert placement. For example, Prophet [194] builds a load balance performance model to estimate the execution time of the MoE layer for each expert placement and then uses a greedy search algorithm to find a well-balanced expert placement. MoE-Prediction [24] traces and analyzes the loads of each expert during the training process, then employs classical prediction algorithms to forecast the expert load proportions, providing valuable guidance for expert placement in MoE training. Lazarus [199] allocates expert replicas using an optimal placement algorithm that assigns more replicas to popular

experts to help balance the load. FlexMoE [126] employs fine-grained replication strategies that select specific heavy experts and duplicate them across multiple devices, dynamically adjusting the expert-to-device mapping at runtime to address routing imbalances. Additional strategies, such as those proposed by MoE-Deploy [65] and Brainstorm [29], rely on historical allocation data and expert reordering to balance loads. Furthermore, techniques like Lynx [56] reduce the number of active experts during batch inference, and BaseLayers [91] formulates token-to-expert allocation as a linear assignment problem to ensure equitable token distribution. Additionally, MoE-ECR [237] allows experts to select the top-k tokens instead of letting each token choose the top-k experts, ensuring that each expert has a fixed bucket size.

4.1.3 All-to-all Communication Optimization. All-to-all communication is a significant bottleneck in MoE layer execution [59, 97, 144], prompting substantial research aimed at optimizing this operation. Many efforts focus on hierarchical communication strategies and data compression techniques to reduce overhead. For instance, Tutel [69], HetuMoE [127], and DeepSpeed-MoE [144] utilize the hierarchical all-to-all algorithm to optimize communication by combining intra-node and inter-node hierarchies. Compared to traditional all-to-all operations, which allow all GPUs to communicate directly with one another, this hierarchical approach first gathers data from all GPUs within a node to a single GPU (intra-node) and then communicates between nodes (inter-node). This method speeds up communication by fully utilizing both levels of bandwidth. Data compression strategies, such as those employed by DeepSpeed-TED [168] and TA-MoE [17], minimize data transfer by eliminating unnecessary information and adapting data volume to the network topology. Innovative paradigms like Janus [107] adopt a data-centric approach, moving experts between devices instead of tokens, thereby significantly reducing communication costs. ExFlow [214] reduces the number of all-to-all operations from two to one by ensuring token context coherence, meaning that all GPUs have the context of all requests rather than only their own. It then exploits inter-layer expert affinity to further reduce the number of all-to-all operations by optimally placing experts in each layer, increasing the probability that tokens remain on local GPUs. Aurora [97] orders token transmission to avoid bandwidth contention, achieving minimal all-to-all communication costs through theoretical derivation. LocMoE [96] and Parm [134] further enhance communication efficiency by converting inter-node communication to intra-node operations and overlapping intra-node with inter-node communications. Additionally, Lina [93] accelerates all-to-all communication by prioritizing all-to-all operations over all-reduce operations.

4.1.4 Task Scheduling. To overlap communication and computation tasks and reduce end-to-end runtime, various task scheduling strategies have been developed. Advanced scheduling strategies leverage architectural and algorithmic innovations to achieve this overlap effectively. For example, ScMoE [13] introduces a shortcut-connected MoE architecture that processes representations from both the current and preceding layers, rather than solely processing representations from the current layer. Specifically, ScMoE employs a top-1 MoE module to handle representations from the preceding layer via a shortcut connection, while a shared expert processes the current layer's representations. This structure effectively decouples communication from its conventional sequence, thereby enhancing the overlap between communication and computation. HiDup [227] splits the input data on each device into two microbatches with no dependencies between them. Consequently, the training of the two microbatches on each device can be carried out in parallel, effectively overlapping the computation of one microbatch with the communication of the other. MoESys [217] employs an elastic MoE training strategy with 2D prefetching and fusion communication over hierarchical storage to overlap computation with the parameter readiness from the hierarchical storage. ScheMoE [166] first modularizes the computing tasks (data compression and expert computation) and communication tasks (collective communication). It then designs

Table 5. Speedup of Parallelism Systems When Compared to DeepSpeed-MoE and FasterMoE in MoE Training

DeepSpeed-MoE [144]			FasterMoE [59]		
Method	Speedup	Metric	Method	Speedup	Metric
HetuMoE [127]	5.88x	Time per iteration	TA-MoE [17]	1.39x	Tokens per second
TA-MoE [17]	1.31x	Tokens per second	PipeMoE [165]	1.69x	Time per iteration
FlexMoE [126]	1.70x	Time to converge	FlexMoE [126]	1.30x	Time to converge
Prophet [194]	2.39x	Time per iteration	Prophet [194]	1.43x	Time per iteration
Parm [134]	3.00x	Time per iteration	ScheMoE [166]	1.25x	Time per iteration
Lazarus [199]	3.40x	Tokens per second	MPMoE [231]	1.60x	Time per iteration
DeepSpeed-TED [168]	1.35x	Time per iteration	SMARTMOE [225]	1.88x	Time per iteration

Table 6. Speedup of Parallelism Systems in MoE Inference

Method	Speedup	Metric	Baseline	Optimized Techniques
Lina [93]	1.63x	Time per batch	DeepSpeed [120]	Prioritize all2all over allreduce
TUTEL [69]	2.03x	Time per batch	Fairseq [149]	Design an identical layout for weights
Aurora [97]	1.81x	Time per batch	Lina [93]	Strategically order token transmissions
ExFlow [214]	2.2x	Tokens per second	Deepspeed-MoE [144]	Reduce two all2all operations into one
Brainstorm [29]	3.33x	Time per batch	Tutel [69]	Preload weight with profiling statics
MoE-Deploy [65]	3.32x	Tokens per second	Tutel [69]	Combine high-used experts with lows
DeepSpeed-MoE [144]	7.25x	Time per batch	PyTorch [5]	Grouped tokens with same data path

an adaptive optimal scheduling algorithm to pipeline the communication and computing tasks based on these modularized operations. Similarly, PipeMoE [165] designs a performance model to predict the computation and communication costs for various MoE workloads and then proposes an optimal polynomial-time solution to pipeline tasks, thereby hiding communication latency based on the performance model's results. Additionally, EPS-MoE [139] dynamically selects optimal kernel implementations to adaptively overlap FFN module computation with all-to-all communication, further improving runtime efficiency.

4.1.5 Performance Summarization. To assess the performance of the methods discussed above, we extract relevant data from their respective papers. Table 5 shows the training speedup of various systems relative to DeepSpeed-MoE and FasterMoE, both of which are widely recognized by researchers and serve as excellent starting points for related studies. Although different systems evaluate their methods using various configurations, a rough comparison can still be made due to the shared baselines. For instance, HetuMoE demonstrates exceptional performance, with the highest speedup compared to other systems. In addition to training speedup, we also report inference speedup and highlight the main optimization techniques of several systems in Table 6. However, comparing performance across systems is challenging due to differences in their baselines. Beyond speedup, some systems offer additional benefits, such as reduced memory usage. As shown in Table 7, many systems focus on optimizing memory usage, which is critical given the large size of MoE models. Additionally, some studies also optimize GPU utilization and communication efficiency.

4.2 Expert Offloading

When deploying MoE models on edge devices, parameter-offloading techniques offer a potential solution to mitigate the challenge of insufficient GPU memory for storing all model parameters. However, traditional parameter-offloading methods load model parameters layer by layer from CPU memory or SSD, neglecting the sparse activation characteristics of MoE models. This oversight incurs significant overhead in parameter loading, leading to suboptimal inference performance.

Table 7. Additional Benefits of Existing Systems

Method	Benefits	Baseline
Lina [93]	Gain 2.3x reduction in all2all communication	DeepSpeed [120]
Prophet [194]	Improve the load balance by 1.75x to 12.06x	FasterMoE [59]
ScheMoE [166]	Gain 1.4x to 2x speedup in all2all communication	NCCLA2A [127], 2DH-A2A [69]
MPMoE [231]	Achieve a 53% decrease in memory usage	FasterMoE [59]
MoESys [217]	Achieve a 18% decrease in memory usage	DeepSpeed [120]
Aurora [97]	Achieve a 1.57x to 1.72x increase in GPU utilization	Lina [93]
MoE-Deploy [65]	Achieve a 79.6% decrease in memory usage	Tutel [69]
DeepSpeed-TED [168]	Support 1.09x to 4.8x larger MoE models	DeepSpeed-MoE [144]

To address this limitation, many studies have proposed expert-offloading techniques, which take advantage of MoE sparse activation patterns. Instead of loading all experts for a given layer, expert-offloading selectively loads only the required experts, thereby significantly reducing loading latency.

As Figure 7(b) shown [177], expert-offloading operates by storing all non-expert weights and a subset of critical experts in GPU memory (referred to as the "expert cache") while offloading less frequently used experts to CPU memory or SSD (referred to as "next-level memory"). When required experts are missing from the expert cache, they are fetched from next-level memory and loaded into the cache, potentially replacing some existing experts. Various studies have optimized this process using different strategies to achieve superior inference performance.

4.2.1 Expert Prefetching. To minimize the waiting time for the required experts, some studies focus on optimizing prefetching techniques that overlap expert loading with GPU computation. Many works predict the experts needed for subsequent layers. For example, Mixtral-Offloading [42], AdapMoE [234], and HOBBIT [177] use current gating inputs to feed the gating module for the following layers, predicting the required experts and prefetching them in advance. This prediction is based on the key observation that the inputs to the gating function in the MoE module share high similarity across successive layers, due to the residual structure of LLMs. As reported in these works, the prediction accuracy can reach about 90%, significantly enhancing the effectiveness of this approach. Pre-gated MoE [70] introduces a pre-gated MoE structure, which identifies the experts required by the next layer during the computation of the current layer and preloads them accordingly. Although this new structure is suitable for prefetching, it may cause a decrease in model accuracy. EdgeMoE [215] constructs a prediction table using a calibrated dataset, leveraging the experts from the current layer to predict the experts for the next layer. This table is built on the observation that the expert activations of sequential layers are statistically correlated. This means that given prior knowledge of the expert activations from layers 1 to n-1, we can estimate the activation probability of each expert at the *n*th layer with high confidence. DyNN-Offload [148] trains a pilot model for each layer to predict the required experts for the next layer. MoE-Infinity [207] tracks the request-level usage frequency of each expert and prefetches high-priority experts based on this frequency. It prefetches experts across multiple layers concurrently, rather than just the next layer, but prioritizes the experts closest to the currently executed layer. ProMoE [171] uses a learned predictor to prefetch experts in a sliding-window manner. This predictor, a small neural network (a two-layer MLP), learns the correlation between layer inputs and expert selections. It can use the inputs of the *i*th layer to predict the expert selections of the (i+k)-th layer, where k is the sliding window size (e.g., 4 or 8). ProMoE runs LLM inference for multiple iterations to collect training data for the predictor in the offline stage and runs the predictor on the CPU to hide its execution latency by overlapping it with the LLM inference during the online stage. Additionally, some works aim to predict all the required experts for the current forward pass at the

start of the forward iteration. ExpertFlow [62] improves MoE inference efficiency by predicting the full routing path before computation begins, enabling accurate expert prefetching and adaptive CPU-GPU caching that minimizes I/O overhead. It further reduces expert activation overhead via dynamic token scheduling, which reorganizes input tokens across batches to maximize expert reuse and lower memory pressure. SiDA [40] employs a network-based hash function to predict the activated experts for each token at each layer for the current incoming batch of data. This predictor is constructed with an LSTM and trained using truncated knowledge distillation. Similarly, Read-ME [12] constructs a pre-gating router decoupled from the MoE backbone to predict all the required experts at once.

4.2.2 Expert Caching. Since GPU memory can only hold a subset of experts (the expert cache), optimizing the cache management policy is crucial for improving the cache hit ratio and reducing transfer time. Several works [42, 65, 70, 215, 219] adopt the **Least Recently Used** (**LRU**) policy, based on the assumption that recently used experts are more likely to be accessed again. According to Mixtral-8x7B [75], when an expert is selected for the ith token, it has a significantly higher probability (more than 10% in most layers) of being selected again for the (i+1)-th token compared to a random selection. In contrast, MoE-Infinity [207] employs a variant of the **Least Frequently Used** (**LFU**) policy to define the caching priority of experts, leveraging its request-level tracking capability. Fiddler [78] maintains a set of important experts in GPU memory using a static dataset to profile the active times of experts, treating these times as a measure of their importance. This method demonstrates an improvement of approximately 3 percentage points in hit rate compared to random placement. Similarly, SwapMoE [86] stores a set of important experts in GPU memory based on a defined expert importance score, but it dynamically updates the cached experts at a suitable frequency. AdapMoE [234] introduces a dynamic cache size for different model layers, which is driven by its adaptive gating algorithm. This algorithm selects a varying number of experts for different layers, thus adjusting the cache size accordingly. Specifically, it assigns a larger cache size to layers that select more experts or experience lower prediction accuracy. HOBBIT [177] proposes a multi-dimensional cache policy that combines LRU, LFU, and a novel **Least High Precision Used** (**LHU**) strategy for its mixed-precision expert cache. Since high-precision experts incur higher loading latencies than low-precision experts, the LHU policy prioritizes high-precision experts to reduce loading costs. Additionally, CacheMoE [169] employs a cache-aware strategy that adjusts the router logits $z = G(x)$ to enhance the likelihood of selecting experts already present in the cache. This approach effectively increases the expert cache hit ratio, thereby reducing the costs associated with expert loading.

4.2.3 Expert Loading. Expert loading time often constitutes the primary bottleneck in expert-offloading inference. As reported in HOBBIT [177], expert loading consumes more than 80% of the total inference time. To address this, several works aim to directly reduce loading costs. For instance, EdgeMoE [215] and HOBBIT [177] reduce loading times by using low-precision experts instead of high-precision ones. Low-precision expert loading requires significantly less time than high-precision experts and can still maintain model accuracy, provided that only the less important experts are replaced. EdgeMoE determines the precision of each expert by profiling its importance using a static dataset. However, this method is highly dependent on the profiled dataset, which reduces its flexibility across different environments and may negatively affect accuracy. To address this limitation, HOBBIT introduces a token-level dynamic expert loading mechanism. This mechanism dynamically selects the appropriate precision for cache-missing experts based on the current inputs, ensuring both accuracy and flexibility. Specifically, it computes the importance score of a cache-missing expert based on the outputs of the gating module. If the score is below a predefined threshold, the system fetches a low-precision version; otherwise, it retrieves the

Table 8. Speedup of Offloading Systems Evaluated on Mixtral-8x7B model [75]

Method	Speedup	Metric	Baseline	Main Techniques
ProMoE [171]	1.16x	Tokens per second	LRU-MoE [171]	Train a predictor to prefetch experts
Fiddler [78]	8.20x	Tokens per second	Mixtral-Offloading [42]	Use CPU to help computation
HOBBIT [177]	2.30x	Tokens per second	MoE-Infinity [207]	Load adaptive precision experts
AdapMoE [234]	1.36x	Latency per sample	Mixtral-Offloading [42]	Skip unimportant experts dynamically
ExpertFlow [62]	1.99x	Tokens per second	Cache-MoE [62]	Train a predictor to prefetch experts
MoE-Infinity [207]	1.96x	Latency per token	Mixtral-Offloading [42]	Fetch experts with request-level tracing
MoE-Lightning [16]	3.50x	Tokens per second	FlexGen [164]	Schedule CPU-GPU-I/O pipeline
Mixtral-Offloading [42]	2.28x	Tokens per second	Accelerate [68]	Use LRU to cache important experts

Table 9. Speedup of Offloading Systems Evaluated on Switch Transformer model [46]

Method	Speedup	Metric	Baseline	Main Techniques
EdgeMoE [215]	2.82x	Latency per token	IO-EXP [215]	Quantize experts into different precisions
SwapMoE [86]	2.00x	Latency per sample	Original-MoE [86]	Update cached experts dynamically
SIDA [40]	3.93x	Samples per second	DeepSpeed [120]	Prefetch experts with a hash function
ExpertFlow [62]	5.23x	Tokens per second	SE-MoE [161]	Train a predictor to prefetch experts
MoE-Infinity [207]	4.53x	Latency per token	Accelerate [68]	Prefetch experts with request-level tracing
Pre-gated MoE [70]	1.50x	Tokens per second	MoE-OnDemand [70]	Design a pre-gate MoE structure

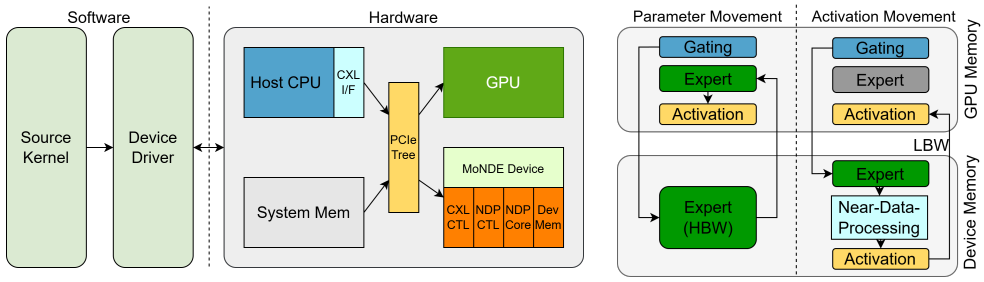
high-precision version to preserve model accuracy. Additionally, AdapMoE [234] employs an adaptive gating algorithm that skips less important experts. This algorithm uses the Fisher information matrix to compute the importance of each expert, further lowering expert loading costs.

4.2.4 CPU Assisting. Beyond relying solely on the GPU for computation, some approaches leverage the CPU to assist in the process. Fiddler [78] performs MoE computation on the CPU by copying intermediate activation values from GPU memory to CPU memory when the required experts are unavailable in GPU memory and then transferring the outputs back to the GPU. This method eliminates the need to load missing experts into GPU memory. As measured, the latency for loading an expert from CPU memory to GPU memory is significantly higher than the combined latency of copying activation values to the CPU, performing expert computation on the CPU, and then copying the results back to the GPU. HOBBIT [177] follows a similar pattern, using the CPU for computation with low-precision experts. Additionally, MoE-Lightning [16] introduces an innovative CPU-GPU-I/O pipelining strategy that enables the simultaneous utilization of both CPU and GPU resources, thus enhancing overall system efficiency. However, this approach is only applicable when CPU resources are sufficiently available. It may not be suitable for platforms with limited CPU resources or shared memory devices, such as the Jetson Orin.

4.2.5 Performance Summarization. Mixtral-8x7B [75] and the Switch Transformer [46] are popular MoE models, and most offloading systems use them as evaluation workloads. We extract relevant performance data from various papers to assess their effectiveness. Table 8 shows that many studies use Mixtral-Offloading as the baseline when evaluating the Mixtral-8x7B model. From the table, it is evident that Fiddler achieves a significant speedup compared to other systems, as it effectively utilizes the CPU to assist computation in GPU-limited environments. However, Table 9 shows that the methods when evaluated on Switch Transformer use different baselines, making direct comparisons difficult. Therefore, for our own studies on this topic, Mixtral-8x7B should stand out as a more suitable workload to consider.

4.3 Framework

Most of the systems mentioned above are built on top of popular deep learning frameworks such as PyTorch [5], Transformers [68], and DeepSpeed [120]. The data is summarized in Table 10. From the table, it is evident that most works in the expert parallelism category implement their systems based



(a) Overview of MoNDE

(b) Parameter and Activation Movement

Fig. 8. (a) MoNDE [82] system design integrating software and hardware. (b) The data flow between AM and PM.

Table 10. Statistics of the Number of Systems Built on Popular Current Frameworks

	Pytorch [5]	Transformers [68]	DeepSpeed [120]	Fairseq [149]	Llama.cpp [53]	vLLM [88]	FasterTransformer [129]
Parallelism System	12	5	9	2	0	0	0
Offloading System	4	7	1	0	2	1	1

on PyTorch, DeepSpeed, and Transformers. PyTorch serves as the foundational deep learning framework, with other libraries like Transformers and DeepSpeed built on top of it. As a result, directly building a system based on PyTorch offers greater flexibility in designing custom functions, though it lacks some popular techniques found in other libraries, such as model parallelism. DeepSpeed provides numerous features to assist in model training, particularly in distributed environments. Transformers, on the other hand, offers many pre-trained models, allowing developers to easily modify the model structure within the library. While this is very convenient for quickly testing ideas, it may not provide the best performance in terms of throughput and latency.

In the expert offloading category, most works still rely on Transformers due to its ease of use. However, some works also utilize Llama.cpp [53], vLLM [88], and FasterTransformers [129]. These three frameworks are optimized for deploying LLMs and offer great performance, but they tend to be more complex to use.

In conclusion, if you are looking to quickly verify an idea, Transformers is an excellent choice. For training models in distributed environments, DeepSpeed is the better option. If your focus is on designing an inference system with high performance, Llama.cpp and vLLM are good starting points. Otherwise, PyTorch remains a solid choice.

5 Hardware-Level Optimization

Recent hardware optimizations for MoE inference have addressed key challenges through novel architectures and co-design approaches. These optimizations target critical issues such as **operations per byte** (**Op/B**) efficiency, heterogeneous computing units, and memory access patterns. The following discusses significant advances in hardware-level solutions.

MoNDE [82] introduces a **near-data processing** (**NDP**) solution to address the challenges of sparse activation and expert parameter transmission overhead (Figure 8). The architecture integrates **Compute Express Link** (**CXL**)-based NDP controllers with specialized NDP cores for in-memory computation, utilizing LPDDR SDRAM (low power double data rate synchronous dynamic random-access memory) for high bandwidth and energy efficiency. The system implements a hybrid computing strategy where GPUs handle frequently accessed "hot" experts while NDP units

process "cold" experts, enabling concurrent execution through an Activation Movement paradigm rather than traditional Parameter Movement pattern.

FLAME [105] is the first accelerating framework that fully exploits MoE sparsity for transformers on FPGA. At the parameter level of the model, it utilizes M:N pruning to reduce unnecessary calculations, which can make a balance between column-balanced structured pruning and unstructured pruning. At the expert level, sparse activation prediction is performed through **circular expert prediction** (**CEPR**). By changing the patterning of the activation path of experts, the accuracy of expert predictions can be effectively improved. Then, using a double buffering mechanism to load the predicted expert while computing the previous expert to improve expert deployment efficiency.

M^3ViT [44] and Edge-MoE [156] constructed their FPGA architecture based on the reordering of attention computation in multitasking scenarios. For inference, M^3ViT can activate only the sparse "expert" pathway relevant to the task of interest for efficiency, and can further achieve zero-overhead switching between tasks with their hardware-level co-design. Edge-MoE is the first end-to-end FPGA implementation for multi-task ViT proposed some aggressive techniques, including an approximate method for solving the excessive complexity of GELU function calculation on FPGA and a unified linear layer module to achieve efficient reuse of hardware resource.

Duplex [223] selects a destination suitable for each layer execution in the device that combines xPU and Logic **processing-in-memory** (**PIM**). That means it could integrate two types of processing units that share device memories. Due to the bottleneck in computation and memory access between these two processing units which can complement each other, high computation and memory access utilization can be achieved simultaneously on the same device. Besides, it introduced an alternative PIM microarchitecture. The logic PIM optimized low Op/B operations through powerful processing units on the logic die, as well as more **through silicon vias** (**TSVs**) to achieve high bandwidth communication between the DRAM die and the logic die. Furthermore, it could execute expert and attention stages in parallel to maximize inference efficiency.

Space-mate [135] has provided its accelerator design for **simultaneous localization and mapping** (**SLAM**) tasks on mobile devices. Mainly including an **Out-of-Order** (**OoO**) SMoE router to alleviate data transactions for low latency, a **single skip** (**SS**) and **dual-skip** (**DS**) heterogeneous core architecture to exploit coarse-grained sparsity caused by similar zero patterns in the same expert for high throughput and energy-efficiency.

6 Future Directions and Open Challenges

Despite significant advances in MoE inference optimization, critical challenges and opportunities exist throughout the computing stack. This section analyzes future research directions along three interconnected dimensions: (1) computing infrastructure, from hardware to system software; (2) key system requirements, including performance, reliability, and efficiency; and (3) the development support ecosystem.

6.1 Computing Infrastructure Optimization

6.1.1 Hardware Integration and Acceleration. Efficient MoE execution demands hardware that transcends traditional dense-computation paradigms [44, 135, 156, 223]. Current platforms struggle with the sparse and dynamic nature of MoE workloads, necessitating specialized hardware solutions [174, 216, 236]. Immediate opportunities include developing specialized circuits for expert routing, creating memory hierarchies optimized for sparse parameter access, and implementing hardware support for dynamic workloads to reduce overhead and improve performance.

Beyond traditional optimizations, emerging platforms offer exciting possibilities. Neuromorphic systems, with their inherent support for sparse, event-driven computation, are a natural fit for accelerating expert activation patterns [158, 159]. Similarly, quantum computing might offer novel

approaches to complex routing decisions, and new memory technologies could provide more efficient storage and access for expert parameters, though significant research challenges remain in bridging classical and quantum computations [103, 132, 172].

6.1.2 System Software Optimization. The system software stack presents significant optimization challenges, as current operating systems and runtimes lack native support for the sparse computation and dynamic scheduling required by MoE models [29]. This gap necessitates fundamental innovations in system software.

Memory management systems require substantial adaptation to accommodate the dynamic expert activation in MoE models. Traditional virtual memory and cache hierarchies, which are optimized for the regular access patterns of dense LLMs, are poorly aligned with the highly irregular and input-dependent memory behavior of MoE architectures [88]. Future work should therefore investigate specialized memory management techniques capable of effectively predicting and prefetching expert parameters, potentially through new abstractions for sparse tensor operations. In distributed environments, resource allocation is another critical area. The dynamic nature of expert activation complicates placement and migration, requiring robust mechanisms for load balancing, fault tolerance, and resource elasticity [24, 194].

Ultimately, the co-design of hardware and software is crucial. Success requires close collaboration between hardware architects and algorithm designers to ensure that optimizations at both layers align with MoE computational requirements, minimizing energy consumption, and data movement while maximizing performance.

6.2 Operational Challenges and Requirements

Deploying and operating MoE systems in production environments introduces complex challenges spanning performance optimization, system reliability, and resource efficiency. These often-conflicting requirements must be carefully balanced for practical adoption across different application domains.

6.2.1 Energy Efficiency and Sustainability. Energy efficiency has received less attention than throughput and latency in MoE research, but the growing environmental impact of AI necessitates making it a primary optimization objective [7, 26, 64, 117, 147]. MoE models present unique energy challenges; while sparse activation reduces computation, overhead from routing, communication, and load balancing can lead to significant energy costs, especially on hardware optimized for dense workloads [69, 126, 128, 166, 240].

Future research should pursue carbon-aware deployment strategies, where expert placement and scheduling decisions consider the carbon intensity of data centers [43, 50, 142]. To support this, comprehensive energy accounting frameworks are essential. Current methods often ignore indirect costs like data movement and cooling; future energy models must capture the full environmental impact to enable truly informed optimization.

6.2.2 Latency and Quality of Service. The dynamic nature of MoE inference makes it difficult to provide consistent latency guarantees and maintain high availability, unlike traditional models with fixed computation paths [29, 207]. A fundamental challenge is the unpredictable, input-dependent expert selection, which causes variability in processing time. Future research must develop techniques to manage this variability, such as advanced caching, workload characterization, and adaptive routing algorithms that balance accuracy with computational overhead [62, 171, 234].

In distributed deployments, system reliability and fault tolerance are also critical [15]. The failure of an expert or communication link can degrade performance and quality. Opportunities exist for developing robust failure detection and recovery mechanisms, graceful degradation strategies, and

redundancy schemes that balance reliability with resource efficiency to maintain service quality under adverse conditions.

6.3 Development Support Ecosystem

6.3.1 Open-Source Frameworks. Current deep learning frameworks like PyTorch [5] and TensorFlow [180] lack native optimizations for MoE workloads, creating a significant barrier to efficient development. To address this, core frameworks require fundamental enhancements, including specialized operators for expert routing, efficient sparse computation primitives, and optimized memory management for expert parameters [111, 139]. These features must be deeply integrated to achieve performance comparable to dense model optimization.

To make MoE development more accessible, high-level APIs and abstractions are also essential. Current implementations often require deep expertise, limiting wider adoption [53]. Future frameworks should provide intuitive APIs for defining experts and routing, alongside seamless integration with the broader ML ecosystem. This involves creating standardized interfaces and compatibility layers so MoE systems can leverage existing tools for training, debugging, and deployment.

6.3.2 Benchmarking and Standardization. The rapid advancement of MoE optimization has created a pressing need for comprehensive benchmarking and standardization [49, 125]. Existing LLM benchmarks are ill-suited for MoE systems, as they fail to account for their unique architectural characteristics. This has led to fragmented evaluation practices that make fair comparisons difficult and impede systematic progress [49].

An effective benchmarking framework must be multi-dimensional, evaluating performance metrics like latency and throughput alongside model quality across various tasks and deployment scenarios. However, developing such standards is challenging due to the dynamic nature of MoE models and the need to balance trade-offs between objectives like quality and efficiency. Looking forward, the field requires standardized benchmark suites, consistent evaluation methodologies, and reference implementations to facilitate comparative analysis and accelerate research through more rigorous and comparable evaluations.

7 Conclusion

This survey provides a comprehensive overview of inference optimization techniques for MoE models across different abstraction levels. We have systematically analyzed various approaches, from model-level optimizations including efficient expert design, compression techniques, and algorithm improvements, to system-level solutions for distributed computing and expert offloading, and finally to hardware-level acceleration designs. Through this analysis, we observe that while MoE models offer a promising approach to scale model capacity with controlled computational costs, their efficient deployment requires careful consideration and optimization at multiple levels of the system stack. The field has seen rapid advancement with numerous innovative solutions, yet several challenges remain, particularly in areas such as hardware integration, energy efficiency, and standardized benchmarking.

Looking forward, we anticipate continued evolution in MoE inference optimization, driven by both academic research and industrial applications. The increasing adoption of MoE architectures in large-scale language models and multimodal systems will likely spur further innovations in optimization techniques. Key areas for future research include the development of specialized hardware architectures for sparse computation, more efficient expert routing algorithms, and improved system-level solutions for distributed deployment. We hope this survey serves as a valuable reference for researchers and practitioners working on MoE deployment, providing a structured understanding of current approaches and inspiring new directions in this

rapidly evolving field. To facilitate ongoing updates and the sharing of cutting-edge advances, we have established a repository at https://github.com/MoE-Inf/awesome-moe-inference/, which we plan to update semiannually to ensure its continued relevance as a dynamic resource for the community.

References

[1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. arXiv:2404.14219. Retrieved from https://arxiv.org/abs/2404.14219 (2024).

[2] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=3zKtaqxLhW

[3] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. arXiv:2508.10925. Retrieved from https://arxiv.org/abs/2508.10925 (2025).

[4] Maryam Akhavan Aghdam, Hongpeng Jin, and Yanzhao Wu. 2024. DA-MoE: Towards dynamic expert allocation for mixture-of-experts models. arXiv:2409.06669. Retrieved from https://arxiv.org/abs/2409.06669

[5] Meta AI. 2024. PyTorch. Retrieved from https://pytorch.org/

[6] Ibrahim M. Alabdulmohsin, Behnam Neyshabur, and Xiaohua Zhai. 2022. Revisiting neural scaling laws in language and vision. *Advances in Neural Information Processing Systems* 35 (2022), 22300–22312.

[7] Negar Alizadeh and Fernando Castor. 2024. Green AI: A preliminary empirical study on energy consumption in DL models across different runtime infrastructures. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*. 134–139.

[8] Anthropic. 2023. The Claude 3 Model Family: Opus, Sonnet, Haiku. Retrieved from https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf

[9] Mikel Artetxe, Shruti Bhosale, Naman Goyal, Todor Mihaylov, Myle Ott, Sam Shleifer, Xi Victoria Lin, Jingfei Du, Srinivasan Iyer, Ramakanth Pasunuru, et al. 2021. Efficient large scale language modeling with mixtures of experts. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 11699–11732.

[10] Guangji Bai, Zheng Chai, Chen Ling, Shiyu Wang, Jiaying Lu, Nan Zhang, Tingwei Shi, Ziyang Yu, Mengdan Zhu, Yifei Zhang, et al. 2024. Beyond efficiency: A systematic survey of resource-efficient large language models. arXiv:2401.00625. Retrieved from https://arxiv.org/abs/2401.00625 (2024).

[11] Baidu-ERNIE-Team. 2025. ERNIE 4.5 Technical Report. Retrieved October 20, 2025 from https://ernie.baidu.com/blog/publication/ERNIE_Technical_Report.pdf

[12] Ruisi Cai, Yeonju Ro, Geon-Woo Kim, Peihao Wang, Babak Ehteshami Bejnordi, Aditya Akella, and Zhangyang Wang. 2024. Read-ME: Refactorizing LLMs as router-decoupled mixture of experts with system co-design. *Advances in Neural Information Processing Systems* 37 (2024), 116126–116148.

[13] Weilin Cai, Juyong Jiang, Le Qin, Junwei Cui, Sunghun Kim, and Jiayi Huang. 2025. Shortcut-connected expert parallelism for accelerating mixture-of-experts. In *Proceedings of the 42nd International Conference on Machine Learning*. PMLR, 267 (2025), 6211–6228.

[14] Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. 2025. A survey on mixture of experts in large language models. *IEEE Transactions on Knowledge and Data Engineering* 37, 7 (2025), 3896–3915.

[15] Weilin Cai, Le Qin, and Jiayi Huang. 2024. MoC-System: Efficient fault tolerance for sparse mixture-of-experts model training. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* 2 (2024), 655–671.

[16] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2024. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* 1 (2024), 715–730.

[17] Chang Chen, Min Li, Zhihua Wu, Dianhai Yu, and Chao Yang. 2022. Ta-moe: Topology-aware large scale mixture-of-expert training. *Advances in Neural Information Processing Systems* 35 (2022), 22173–22186.

[18] I-Chun Chen, Hsu-Shen Liu, Wei-Fang Sun, Chen-Hao Chao, Yen-Chang Hsu, Chun-Yi Lee . 2025. Retraining-free merging of sparse mixture-of-experts via hierarchical clustering. *Forty-second International Conference on Machine Learning*. https://openreview.net/forumid=hslOzRxzXL

[19] Tianyu Chen, Shaohan Huang, Yuan Xie, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. 2022. Task-specific expert pruning for sparse mixture-of-experts. arXiv:2206.00277. Retrieved from https://arxiv.org/abs/2206.00277 (2022).

[20] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. 2023. Reproducible scaling laws for contrastive language-image learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2818–2829.

[21] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.

[22] Mohammed Nowaz Rabbani Chowdhury, Meng Wang, Kaoutar El Maghraoui, Naigang Wang, Pin-Yu Chen, and Christopher Carothers. 2024. A provably effective method for pruning experts in fine-tuned sparse mixture-of-experts. *International Conference on Machine Learning*. 8815–8847.

[23] Mohammed Nowaz Rabbani Chowdhury, Shuai Zhang, Meng Wang, Sijia Liu, and Pin-Yu Chen. 2023. Patch-level routing in mixture-of-experts is provably sample-efficient for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning*. PMLR, 6074–6114.

[24] Peizhuang Cong, Aomufei Yuan, Shimao Chen, Yuxuan Tian, Bowen Ye, and Tong Yang. 2024. Prediction is all MoE needs: Expert load distribution goes from fluctuating to stabilizing. arXiv:2404.16914. Retrieved from https://arxiv.org/abs/2404.16914 (2024).

[25] Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, et al. 2022. No language left behind: Scaling human-centered machine translation. arXiv:2207.04672. Retrieved from https://arxiv.org/abs/2207.04672 (2022).

[26] Luís Cruz, Xavier Franch Gutierrez, and Silverio Martínez-Fernández. 2025. Innovating for tomorrow: The convergence of SE and green AI. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–3.

[27] Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D. Manning. 2024. MoEUT: Mixture-of-experts universal transformers. *Advances in Neural Information Processing Systems* 37 (2024), 28589–28614.

[28] Róbert Csordás, Piotr Piękos, Kazuki Irie, and Jürgen Schmidhuber. 2024. Switchhead: Accelerating transformers with mixture-of-experts attention. *Advances in Neural Information Processing Systems* 37 (2024), 74411–74438.

[29] Weihao Cui, Zhenhua Han, Lingji Ouyang, Yichuan Wang, Ningxin Zheng, Lingxiao Ma, Yuqing Yang, Fan Yang, Jilong Xue, Lili Qiu, et al. 2023. Optimizing dynamic neural networks with brainstorm. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 797–815. Retrieved from https://www.usenix.org/conference/osdi23/presentation/cui

[30] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* 1 (2024), 1280–1297.

[31] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948. Retrieved from https://arxiv.org/abs/2501.12948

[32] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, et al. 2024. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv:2405.04434. Retrieved from https://arxiv.org/abs/2405.04434

[33] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, et al. 2024. DeepSeek-V3 technical report. arXiv:2412.19437. Retrieved from https://arxiv.org/abs/2412.19437 (2024).

[34] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. 2023. Scaling vision transformers to 22 billion parameters. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 7480–7512. Retrieved from https://proceedings.mlr.press/v202/dehghani23a.html

[35] Yifeng Ding, Jiawei Liu, Yuxiang Wei, Terry Yue Zhuo, and Lingming Zhang. 2024. XFT: Unlocking the power of code instruction tuning by simply merging upcycled mixture-of-experts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* 1 (2024), 12941–12955.

[36] Bill Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the 3rd international workshop on paraphrasing (IWP2005)*.

[37] Alexey Dosovitskiy. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*. arXiv:2010.11929. Retrieved from https://arxiv.org/abs/2010.11929 (2020).

[38] Shihan Dou, Enyu Zhou, Yan Liu, Songyang Gao, Wei Shen, Limao Xiong, Yuhao Zhou, Xiao Wang, Zhiheng Xi, Xiaoran Fan, et al. 2024. LoRAMoE: Alleviating world knowledge forgetting in large language models via MoE-style plugin. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1932–1945.

[39] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the International Conference on Machine Learning*. PMLR, 5547–5569.

[40] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Li, and Yiran Chen. 2024. SiDA: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models. *Proceedings of Machine Learning and Systems* 6 (2024), 224–238.

[41] David Eigen, Marc'Aurelio Ranzato, and Ilya Sutskever. 2013. Learning factored representations in a deep mixture of experts. arXiv:1312.4314. Retrieved from https://arxiv.org/abs/1312.4314 (2013). Retrieved from https://api.semanticscholar.org/CorpusID:11492613

[42] Artyom Eliseev and Denis Mazur. 2023. Fast inference of mixture-of-experts language models with offloading. arXiv:2312.17238. Retrieved from https://arxiv.org/abs/2312.17238 (2023).

[43] Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Osi, Prateek Sharma, Fan Chen, and Lei Jiang. 2023. Llmcarbon: Modeling the end-to-end carbon footprint of large language models. *The Twelfth International Conference on Learning Representations*. arXiv:2309.14393. Retrieved from https://arxiv.org/abs/2309.14393 (2023).

[44] Zhiwen Fan, Rishov Sarkar, Ziyu Jiang, Tianlong Chen, Kai Zou, Yu Cheng, Cong Hao, Zhangyang Wang, Hanxue Liang . 2022. M3ViT: Mixture-of-experts vision transformer for efficient multi-task learning with model-accelerator co-design. *Advances in Neural Information Processing Systems* 35 (2022), 28441–28457.

[45] William Fedus, Jeff Dean, and Barret Zoph. 2022. A review of sparse expert models in deep learning. arXiv:2209.01667. Retrieved from https://arxiv.org/abs/2209.01667 (2022).

[46] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[47] Elias Frantar and Dan Alistarh. 2023. Qmoe: Practical sub-1-bit compression of trillion-parameter models. In *Proceedings of the 5th MLSys Conference*. arXiv:2310.16795. Retrieved from https://arxiv.org/abs/2310.16795 (2023).

[48] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *The Eleventh International Conference on Learning Representations*. https://openreview.net/forumid=tcbBPnfwxS

[49] Yao Fu, Yinsicheng Jiang, Yeqi Huang, Ping Nie, Zhan Lu, Leyang Xue, Congjie He, Man-Kit Sit, Jilong Xue, Li Dong, et al. 2024. MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems. arXiv:2412.07067. Retrieved from https://arxiv.org/abs/2412.07067 (2024).

[50] Zhenxiao Fu, Fan Chen, Shan Zhou, Haitong Li, and Lei Jiang. 2025. LLMCO2: Advancing accurate carbon footprint prediction for LLM inferences. *ACM SIGENERGY Energy Informatics Review* 5, 2 (2025), 63–68.

[51] Chongyang Gao, Kezhen Chen, Jinmeng Rao, Baochen Sun, Ruibo Liu, Daiyi Peng, Yawen Zhang, Xiaoyuan Guo, Jie Yang, and VS Subrahmanian. 2024. Higher layers need more lora experts. arXiv:2402.08562. Retrieved from https://arxiv.org/abs/2402.08562 (2024).

[52] Ze-Feng Gao, Peiyu Liu, Wayne Xin Zhao, Zhong-Yi Lu, and Ji-Rong Wen. 2022. Parameter-efficient mixture-of-experts architecture for pre-trained language models. In *Proceedings of the 29th International Conference on Computational Linguistics*. 3263–3273.

[53] Georgi Gerganov. 2023. Llama.cpp. Retrieved from https://github.com/ggerganov/llama.cpp

[54] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2023. Knowledge distillation of large language models. *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id5h0qf7IBZZ

[55] Yongxin Guo, Zhenglin Cheng, Xiaoying Tang, Zhaopeng Tu, and Tao Lin. 2024. Dynamic mixture of experts: An auto-tuning approach for efficient transformer models. *The Thirteenth International Conference on Learning Representations*. arXiv:2405.14297. Retrieved from https://arxiv.org/abs/2405.14297 (2024).

[56] Vima Gupta, Kartik Sinha, Ada Gavrilovska, and Anand Padmanabha Iyer. 2024. Lynx: Enabling efficient MoE inference through dynamic batch-aware expert selection. arXiv:2411.08982. Retrieved from https://arxiv.org/abs/2411.08982 (2024).

[57] J. B. Hampshire and A. Waibel. 1992. The Meta-Pi network: Building distributed knowledge representations for robust multisource pattern recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14, 7 (1992), 751–769. DOI: https://doi.org/10.1109/34.142911

[58] Jiaao He, Jiezhong Qiu, Aohan Zeng, Zhilin Yang, Jidong Zhai, and Jie Tang. 2021. Fastmoe: A fast mixture-of-expert training system. arXiv:2103.13262. Retrieved from https://arxiv.org/abs/2103.13262 (2021).

[59] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Seoul, Republic of Korea) *(PPoPP '22)*. Association for Computing Machinery, New York, NY, USA, 120–134. DOI: https://doi.org/10.1145/3503221.3508418

[60] Shwai He, Daize Dong, Liang Ding, and Ang Li. 2024. Demystifying the compression of mixture-of-experts through a unified framework. arXiv:2406.02500. Retrieved from https://arxiv.org/abs/2406.02500 (2024).

[61] Shwai He, Run-Ze Fan, Liang Ding, Li Shen, Tianyi Zhou, and Dacheng Tao. 2023. Merging experts into one: Improving computational efficiency of mixture of experts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 14685–14691.

[62] Xin He, Shunkang Zhang, Yuxin Wang, Haiyan Yin, Zihao Zeng, Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Ivor Tsang, and Ong Yew Soon. 2024. ExpertFlow: Optimized expert activation and token allocation for efficient mixture-of-experts inference. arXiv:2410.17954. Retrieved from https://arxiv.org/abs/2410.17954 (2024).

[63] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *International Conference on Learning Representations*. https://openreview.net/forum?id=d7KBjmI3GmQ

[64] Xiaofeng Hou, Cheng Xu, Chao Li, Jiacheng Liu, Xuehan Tang, Kwang-Ting Cheng, and Minyi Guo. 2024. Improving efficiency in multi-modal autonomous embedded systems through adaptive gating. *IEEE Transactions on Computers* 74, 2 (2024), 691–704.

[65] Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Hsien-Hsin S. Lee, Anjali Sridhar, Shruti Bhosale, Carole-Jean Wu, and Benjamin Lee. 2023. Towards MoE deployment: Mitigating inefficiencies in mixture-of-expert (MoE) inference. arXiv:2303.06182. Retrieved from https://arxiv.org/abs/2303.06182 (2023).

[66] Wei Huang, Yue Liao, Jianhui Liu, Ruifei He, Haoru Tan, Shiming Zhang, Hongsheng Li, Si Liu, and Xiaojuan Qi. 2024. MC-MoE: Mixture compressor for mixture-of-experts LLMs gains more. *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forumid=hheFYjOsWO

[67] Yongqi Huang, Peng Ye, Xiaoshui Huang, Sheng Li, Tao Chen, Tong He, and Wanli Ouyang. 2023. Experts weights averaging: A new general training scheme for vision transformers. arXiv:2308.06093. Retrieved from https://arxiv.org/abs/2308.06093 (2023).

[68] Huggingface. 2023. Transformers: State-of-the-art Machine Learning for JAX, PyTorch and TensorFlow. Retrieved from https://github.com/huggingface/transformers

[69] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. 2023. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems* 5 (2023), 269–287.

[70] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031.

[71] Shibal Ibrahim, Wenyu Chen, Hussein Hazimeh, Natalia Ponomareva, Zhe Zhao, and Rahul Mazumder. 2023. Comet: Learning cardinality constrained mixture of experts with trees and local search. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 832–844.

[72] HamidReza Imani, Abdolah Amirany, and Tarek El-Ghazawi. 2024. Mixture of experts with mixture of precisions for tuning quality of service. *IEEE International Conference on Rebooting Computing (ICRC'24)*. 1–6.

[73] Robert Jacobs, Michael Jordan, Steven Nowlan, and Geoffrey Hinton. 1991. Adaptive mixture of local expert. *Neural Computation* 3, 1 (1991), 79–87. DOI: https://doi.org/10.1162/neco.1991.3.1.79

[74] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond data and model parallelism for deep neural networks. *Proceedings of Machine Learning and Systems* 1 (2019), 1–13.

[75] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. arXiv:2401.04088. Retrieved from https://arxiv.org/abs/2401.04088 (2024).

[76] Peng Jin, Bo Zhu, Li Yuan, and Shuicheng Yan. 2024. Moe++: Accelerating mixture-of-experts methods with zero-computation experts. *International Conference on Learning Representations* 2025 (2024), 50832–50856. https://proceedings.iclr.cc/paper_files/paper/2025/file/7efe88bb4138d602e56637cfcf713654-Paper-Conference.pdf

[77] Peng Jin, Bo Zhu, Li Yuan, and Shuicheng Yan. 2025. MoH: Multi-head attention as mixture-of-head attention. *Forty-second International Conference on Machine Learning*. https://openreview.net/forumid=eYtgs9k75o

[78] Keisuke Kamahori, Yile Gu, Kan Zhu, and Baris Kasikci. 2024. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forumid=N5fVv6PZGz

[79] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. arXiv:2001.08361. Retrieved from https://arxiv.org/abs/2001.08361 (2020).

[80] Daniel Khashabi, Snigdha Chaturvedi, Michael Roth, Shyam Upadhyay, and Dan Roth. 2018. Looking beyond the surface: A challenge set for reading comprehension over multiple sentences. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 252–262.

[81] Sungyoon Kim, Youngjun Kim, Kihyo Moon, and Minsung Jang. 2024. Ladimo: Layer-wise distillation inspired moefier. arXiv:2408.04278. Retrieved from https://arxiv.org/abs/2408.04278 (2024).

[82] Taehyun Kim, Kwanseok Choi, Youngmock Cho, Jaehoon Cho, Hyuk-Jae Lee, and Jaewoong Sim. 2024. MoNDE: Mixture of near-data experts for large-scale sparse models. In *Proceedings of the 61st ACM/IEEE Design Automation*

Conference (San Francisco, CA, USA) *(DAC '24)*. Association for Computing Machinery, New York, NY, USA, Article 221, 6 pages. DOI: https://doi.org/10.1145/3649329.3655951

[83] Young Jin Kim, Raffy Fahim, and Hany Hassan Awadalla. 2023. Mixture of quantized experts (MoQE): Complementary effect of low-bit quantization and robustness. arXiv:2310.02410. Retrieved from https://arxiv.org/abs/2310.02410 (2023).

[84] Young Jin Kim, Rawn Henry, Raffy Fahim, and Hany Hassan Awadalla. 2022. Who says elephants can't run: Bringing large scale MoE models into cloud scale production. In *Proceedings of the Third Workshop on Simple and Efficient Natural Language Processing (SustaiNLP)*. 36–43.

[85] Kimi Team. 2025. Kimi K2: Open Agentic Intelligence. Retrieved October 20, 2025 from https://github.com/MoonshotAI/Kimi-K2/blob/main/tech_report.pdf

[86] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Linghe Kong, and Yunxin Liu. 2023. Serving MoE models on resource-constrained edge devices via dynamic expert swapping. *IEEE Transactions on Computers* 74, 8 (2025), 2799–2811.

[87] Alex Krizhevsky. 2009. Learning multiple layers of features from tiny images. *Master's thesis, University of Tront* (2009).

[88] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

[89] Jaeseong Lee, Aurick Qiao, Daniel F. Campos, Zhewei Yao, Yuxiong He, and Seung-won hwang.. 2025. STUN: Structured-then-unstructured pruning for scalable MoE pruning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* 1 (2025), 13660–13676.

[90] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. Gshard: Scaling giant models with conditional computation and automatic sharding. *International Conference on Learning Representations*. https://openreview.net/forumid=qrwe7XHTmYb

[91] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. 2021. Base layers: Simplifying training of large, sparse models. In *Proceedings of the International Conference on Machine Learning*. PMLR, 6265–6274.

[92] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. Llm inference serving: Survey of recent advances and opportunities. *IEEE High Performance Extreme Computing Conference*. 1–8.

[93] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating distributed MoE training and inference with lina. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 945–959. Retrieved from https://www.usenix.org/conference/atc23/presentation/li-jiamin

[94] Junnan Li, Ramprasaath Selvaraju, Akhilesh Gotmare, Shafiq Joty, Caiming Xiong, and Steven Chu Hong Hoi. 2021. Align before fuse: Vision and language representation learning with momentum distillation. *Advances in Neural Information Processing Systems* 34 (2021), 9694–9705.

[95] Jiamin Li, Qiang Su, Yitao Yang, Yimin Jiang, Cong Wang, and Hong Xu. 2023. Adaptive gating in mixture-of-experts based language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3577–3587.

[96] Jing Li, Zhijie Sun, Xuan He, Li Zeng, Yi Lin, Entong Li, Binfan Zheng, Rongqian Zhao, and Xin Chen. 2024. Locmoe: A low-overhead moe for large language model training. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 6377–6387.

[97] Jialong Li, Shreyansh Tripathi, Lakshay Rastogi, Yiming Lei, Rui Pan, and Yiting Xia. 2025. Optimizing mixture-of-experts inference time combining model deployment and communication scheduling. *IEEE Transactions on Networking* 34 (2025), 2478–2497.

[98] Jinhao Li, Jiaming Xu, Shan Huang, Yonghua Chen, Wen Li, Jun Liu, Yaoxiu Lian, Jiayi Pan, Li Ding, Hao Zhou, et al. 2024. Large language model inference acceleration: A comprehensive hardware perspective. arXiv:2410.04466. Retrieved from https://arxiv.org/abs/2410.04466 (2024).

[99] Margaret Li, Suchin Gururangan, Tim Dettmers, Mike Lewis, Tim Althoff, Noah A. Smith, and Luke Zettlemoyer. 2022. Branch-train-merge: Embarrassingly parallel training of expert language models. arXiv:2208.03306. Retrieved from https://arxiv.org/abs/2208.03306 (2022).

[100] Pingzhi Li, Xiaolong Jin, Yu Cheng, and Tianlong Chen. 2024. Examining post-training quantization for mixture-of-experts: A benchmark. arXiv:2406.08155. Retrieved from https://arxiv.org/abs/2406.08155 (2024).

[101] Pingzhi Li, Zhenyu Zhang, Prateek Yadav, Yi-Lin Sung, Yu Cheng, Mohit Bansal, and Tianlong Chen. 2024. Merge, then compress: Demystify efficient SMoe with hints from its routing policy. *The Twelfth International Conference on Learning Representations*. https://openreview.net/forumid=eFWG9Cy3WK

[102] Yixiao Li, Yifan Yu, Qingru Zhang, Chen Liang, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2023. Losparse: Structured compression of large language models based on low-rank and sparse approximation. In *Proceedings of the International Conference on Machine Learning*. PMLR, 20336–20350.

[103] Zhiding Liang, Jinglei Cheng, Rui Yang, Hang Ren, Zhixin Song, Di Wu, Xuehai Qian, Tongyang Li, and Yiyu Shi. 2023. Unleashing the potential of llms for quantum computing: A study in quantum architecture design. arXiv:2307.08191. Retrieved from https://arxiv.org/abs/2307.08191 (2023).

[104] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, et al. 2024. Jamba: A hybrid transformer-mamba language model. arXiv:2403.19887. Retrieved from https://arxiv.org/abs/2403.19887 (2024).

[105] Xuanda Lin, Huinan Tian, Wenxiao Xue, Lanqi Ma, Jialin Cao, Manting Zhang, Jun Yu, and Kun Wang. 2024. FLAME: Fully leveraging MoE sparsity for transformer on FPGA. In *Proceedings of the 61st ACM/IEEE Design Automation Conference* (San Francisco, CA, USA) *(DAC '24)*. Association for Computing Machinery, New York, NY, USA, Article 248, 6 pages. DOI: https://doi.org/10.1145/3649329.3656507

[106] Enshu Liu, Junyi Zhu, Zinan Lin, Xuefei Ning, Matthew B. Blaschko, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. 2024. Efficient expert pruning for sparse mixture-of-experts language models: Enhancing performance and reducing inference costs. arXiv:2407.00945. Retrieved from https://arxiv.org/abs/2407.00945 (2024).

[107] Juncai Liu, Jessie Hui Wang, and Yimin Jiang. 2023. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) *(ACM SIGCOMM'23)*. Association for Computing Machinery, New York, NY, USA, 486–498. DOI: https://doi.org/10.1145/3603269.3604869

[108] Liyuan Liu, Young Jin Kim, Shuohang Wang, Chen Liang, Yelong Shen, Hao Cheng, Xiaodong Liu, Masahiro Tanaka, Xiaoxia Wu, Wenxiang Hu, et al. 2024. GRIN: GRadient-INformed MoE. arXiv:2409.12136. Retrieved from https://arxiv.org/abs/2409.12136 (2024).

[109] Mengfan Liu, Wei Wang, and Chuan Wu. 2025. Optimizing distributed deployment of mixture-of-experts model inference in serverless computing. *IEEE Infocom 2025-IEEE Conference on Computer Communications*. 1–10.

[110] Qidong Liu, Xian Wu, Xiangyu Zhao, Yuanshao Zhu, Derong Xu, Feng Tian, and Yefeng Zheng. 2024. When MOE meets LLMs: Parameter efficient fine-tuning for multi-task medical applications. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1104–1114.

[111] Rui Liu, Young Jin Kim, Alexandre Muzio, and Hany Hassan. 2022. Gating dropout: Communication-efficient regularization for sparsely activated transformers. In *Proceedings of the International Conference on Machine Learning*. PMLR, 13782–13792.

[112] Zefang Liu and Jiahua Luo. 2024. AdaMoLE: Fine-tuning large language models with adaptive mixture of low-rank adaptation experts. *First Conference on Language Modeling*. https://openreview.net/forumid=ndY9qFf9Sa

[113] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, et al. 2023. The flan collection: Designing data and methods for effective instruction tuning. In *Proceedings of the International Conference on Machine Learning*. PMLR, 22631–22648.

[114] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. 2024. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* 1 (2024), 6159–6172.

[115] Tongxu Luo, Jiahe Lei, Fangyu Lei, Weihao Liu, Shizhu He, Jun Zhao, and Kang Liu. 2024. Moelora: Contrastive learning guided mixture of experts on parameter-efficient fine-tuning for large language models. arXiv:2402.12851. Retrieved from https://arxiv.org/abs/2402.12851 (2024).

[116] Zixuan Ma, Jiaao He, Jiezhong Qiu, Huanqi Cao, Yuanwei Wang, Zhenbo Sun, Liyan Zheng, Haojie Wang, Shizhi Tang, Tianyu Zheng, et al. 2022. BaGuaLu: Targeting brain scale pretrained models with over 37 million cores. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Seoul, Republic of Korea) *(PPoPP '22)*. Association for Computing Machinery, New York, NY, USA, 192–204. DOI: https://doi.org/10.1145/3503221.3508417

[117] Yuyi Mao, Xianghao Yu, Kaibin Huang, Ying-Jun Angela Zhang, and Jun Zhang. 2024. Green edge AI: A contemporary survey. *Proceedings of the IEEE* 112, 7 (2024), 880–911.

[118] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2025. Shortgpt: Layers in large language models are more redundant than you expect. In *Findings of the Association for Computational Linguistics: ACL 2025*. 20192–20204.

[119] Meta AI. 2025. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. Retrieved October 20, 2025 from https://ai.meta.com/blog/llama-4-multimodal-intelligence/

[120] Microsoft. 2023. DeepSpeed. RetrievedfromhttPs://github.com/microsoft/DeepSpeed

[121] MiniMax, Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, et al.2025. MiniMax-01: Scaling foundation models with lightning attention. arXiv:2501.08313. Retrieved from https://arxiv.org/abs/2501.08313

[122] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. 2024. OLMoE: Open mixture-of-experts language models. arXiv:2409.02060. Retrieved from https://arxiv.org/abs/2409.02060 (2024).

[123] Alexandre Muzio, Alex Sun, and Churan He. 2024. SEER-MoE: Sparse expert efficiency through regularization for mixture-of-experts. arXiv:2404.05089. Retrieved from https://arxiv.org/abs/2404.05089 (2024).

[124] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.

[125] Nam V. Nguyen, Thong T. Doan, Luong Tran, Van Nguyen, and Quang Pham. 2024. LIBMoE: A library for comprehensive benchmarking mixture of experts in large language models. arXiv:2411.00918. Retrieved from https://arxiv.org/abs/2411.00918 (2024).

[126] Xiaonan Nie, Xupeng Miao, Zilong Wang, Zichao Yang, Jilong Xue, Lingxiao Ma, Gang Cao, and Bin Cui. 2023. Flexmoe: Scaling large-scale sparse pre-trained model training via dynamic device placement. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–19.

[127] Xiaonan Nie, Pinxue Zhao, Xupeng Miao, Tong Zhao, and Bin Cui. 2022. HetuMoE: An efficient trillion-scale mixture-of-expert distributed training system. arXiv:2203.14685. Retrieved from https://arxiv.org/abs/2203.14685 (2022).

[128] Shupeng Ning, Hanqing Zhu, Chenghao Feng, Jiaqi Gu, Zhixing Jiang, Zhoufeng Ying, Jason Midkiff, Sourabh Jain, May H. Hlaing, David Z. Pan, et al. 2024. Photonic-electronic integrated circuits for high-performance computing and ai accelerators. *Journal of Lightwave Technology* 42, 22 (2024), 7834–7859.

[129] NVIDIA. 2019. FasterTransformer. Retrieved from https://github.com/NVIDIA/FasterTransformer

[130] OpenAI. 2023. Gpt-4 technical report. arXiv:2303.08774. Retrieved from https://arxiv.org/abs/2303.08774 (2023).

[131] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.

[132] Harish Padmanaban. 2024. Quantum computing and AI in the cloud. *Journal of Computational Intelligence and Robotics* 4, 1 (2024), 14–32.

[133] Bowen Pan, Yikang Shen, Haokun Liu, Mayank Mishra, Gaoyuan Zhang, Aude Oliva, Colin Raffel, and Rameswar Panda. 2024. Dense training, sparse inference: Rethinking training of mixture-of-experts language models. arXiv:2404.05567. Retrieved from https://arxiv.org/abs/2404.05567 (2024).

[134] Xinglin Pan, Wenxiang Lin, Shaohuai Shi, Xiaowen Chu, Weinong Sun, and Bo Li. 2024. Parm: Efficient training of large sparsely-activated models with dedicated schedules. In *Proceedings of the IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. 1880–1889. DOI: https://doi.org/10.1109/INFOCOM52122.2024.10621327

[135] Gwangtae Park, Seokchan Song, Haoyang Sang, Dongseok Im, Donghyeon Han, Sangyeob Kim, Hongseok Lee, and Hoi-Jun Yoo. 2024. 20.8 Space-Mate: A 303.5mW Real-Time Sparse Mixture-of-Experts-Based NeRF-SLAM Processor for Mobile Spatial Computing. In *Proceedings of the 2024 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 67. 374–376. DOI: https://doi.org/10.1109/ISSCC49657.2024.10454487

[136] Sejik Park. 2024. Learning more generalized experts by merging experts in mixture-of-experts. arXiv:2405.11530. Retrieved from https://arxiv.org/abs/2405.11530 (2024).

[137] Hao Peng, Roy Schwartz, Dianqi Li, and Noah A. Smith. 2020. A mixture of h - 1 Heads is better than h heads. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 6566–6577. DOI: https://doi.org/10.18653/v1/2020.acl-main.587

[138] Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. 2023. From sparse to soft mixtures of experts. arXiv:2308.00951. Retrieved from https://arxiv.org/abs/2308.00951 (2023).

[139] Yulei Qian, Fengcun Li, Xiangyang Ji, Xiaoyu Zhao, Jianchao Tan, Kefeng Zhang, and Xunliang Cai. 2024. EPS-MoE: Expert pipeline scheduler for cost-efficient MoE inference. arXiv:2410.12247. Retrieved from https://arxiv.org/abs/2410.12247 (2024).

[140] Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 1 (2025), 5005–5018. https://aclanthology.org/2025.acl-long.249/

[141] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning*. PMLR, 8748–8763.

[142] Ana Radovanović, Ross Koningstein, Ian Schneider, Bokan Chen, Alexandre Duarte, Binz Roy, Diyue Xiao, Maya Haridasan, Patrick Hung, Nick Care, et al. 2022. Carbon-aware computing for datacenters. *IEEE Transactions on Power Systems* 38, 2 (2022), 1270–1280.

[143] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.

[144] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (Eds.). PMLR, 18332–18346. Retrieved from https://proceedings.mlr.press/v162/rajbhandari22a.html

[145] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *Proceedings of the SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

[146] P Rajpurkar. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2383–2392. https://aclanthology.org/D16-1264/

[147] Raghu Raman, Debidutta Pattnaik, Hiran H. Lathabai, Chandan Kumar, Kannan Govindan, and Prema Nedungadi. 2024. Green and sustainable AI research: An integrated thematic and topic modeling analysis. *Journal of Big Data* 11, 1 (2024), 55.

[148] Jie Ren, Dong Xu, Shuangyan Yang, Jiacheng Zhao, Zhicheng Li, Christian Navasca, Chenxi Wang, Harry Xu, and Dong Li. 2024. Enabling large dynamic neural network training with learning-based memory management. In *Proceedings of the 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 788–802. DOI: https://doi.org/10.1109/HPCA57654.2024.00066

[149] Facebook AI Research. 2019. Fairseq. Retrieved from https://github.com/facebookresearch/fairseq

[150] Snowflake AI Research. 2024. Snowflake Arctic: The Best LLM for Enterprise AI – Efficiently Intelligent, Truly Open. Retrieved from https://www.snowflake.com/en/blog/arctic-open-efficient-foundation-language-models-snowflake/

[151] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S Gordon. 2011. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *Proceedings of the 2011 AAAI spring symposium series*.

[152] Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. 2021. Hash layers for large sparse models. *Advances in Neural Information Processing Systems* 34 (2021), 17555–17566.

[153] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. 2015. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision* 115 (2015), 211–252.

[154] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM* 64, 9 (2021), 99–106.

[155] Felipe Cruz Salinas, Kenichi Kumatani, Robert Gmyr, Linquan Liu, and Yu Shi. 2022. *Knowledge distillation for mixture of experts models in speech recognition*. Technical Report. Technical Report MSR-TR-2022-6, Microsoft Research, May 2022. Retrieved from https://www-$\protect\TU\textellipsis$

[156] Rishov Sarkar, Hanxue Liang, Zhiwen Fan, Zhangyang Wang, and Cong Hao. 2023. Edge-MoE: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts. In *Proceedings of the 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 01–09. DOI: https://doi.org/10.1109/ICCAD57390.2023.10323651

[157] Soumajyoti Sarkar, Leonard Lausen, Volkan Cevher, Sheng Zha, Thomas Brox, and George Karypis. 2024. Revisiting SMoE language models by evaluating inefficiencies with task specific expert pruning. arXiv:2409.01483. Retrieved from https://arxiv.org/abs/2409.01483 (2024).

[158] Catherine D. Schuman, Shruti R. Kulkarni, Maryam Parsa, J. Parker Mitchell, Prasanna Date, and Bill Kay. 2022. Opportunities for neuromorphic computing algorithms and applications. *Nature Computational Science* 2, 1 (2022), 10–19.

[159] Catherine D. Schuman, Thomas E. Potok, Robert M. Patton, J. Douglas Birdwell, Mark E. Dean, Garrett S. Rose, and James S. Plank. 2017. A survey of neuromorphic computing and neural networks in hardware. arXiv:1705.06963. Retrieved from https://arxiv.org/abs/1705.06963 (2017).

[160] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations*. https://openreview.net/forumid=B1ckMDqlg

[161] Liang Shen, Zhihua Wu, WeiBao Gong, Hongxiang Hao, Yangfan Bai, HuaChao Wu, Xinxuan Wu, Jiang Bian, Haoyi Xiong, Dianhai Yu, et al. 2022. Se-moe: A scalable and efficient mixture-of-experts distributed training and inference system. arXiv:2205.10034. Retrieved from https://arxiv.org/abs/2205.10034 (2022).

[162] Yikang Shen, Zhen Guo, Tianle Cai, and Zengyi Qin. 2024. Jetmoe: Reaching llama2 performance with 0.1 m dollars. arXiv:2404.07413. Retrieved from https://arxiv.org/abs/2404.07413 (2024).

[163] Yikang Shen, Zheyu Zhang, Tianyou Cao, Shawn Tan, Zhenfang Chen, and Chuang Gan. 2023. Moduleformer: Learning modular large language models from uncurated data. arXiv:2306.04640. Retrieved from https://arxiv.org/abs/2306.04640 (2023).

[164] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *Proceedings of the International Conference on Machine Learning*. PMLR, 31094–31116.

[165] Shaohuai Shi, Xinglin Pan, Xiaowen Chu, and Bo Li. 2023. PipeMoE: Accelerating mixture-of-experts through adaptive pipelining. In *Proceedings of the IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. 1–10. DOI: https://doi.org/10.1109/INFOCOM53939.2023.10228874

[166] Shaohuai Shi, Xinglin Pan, Qiang Wang, Chengjian Liu, Xiaozhe Ren, Zhongzhe Hu, Yu Yang, Bo Li, and Xiaowen Chu. 2024. ScheMoE: An extensible mixture-of-experts distributed training system with tasks scheduling. In *Proceedings of the 19th European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 236–249. DOI: https://doi.org/10.1145/3627703.3650083

[167] Fangxun Shu, Yue Liao, Le Zhuo, Chenning Xu, Lei Zhang, Guanghao Zhang, Haonan Shi, Long Chen, Tao Zhong, Wanggui He, et al. 2025. Llava-mod: Making llava tiny via moe knowledge distillation. *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forumid=uWtLOy35WD

[168] Siddharth Singh, Olatunji Ruwase, Ammar Ahmad Awan, Samyam Rajbhandari, Yuxiong He, and Abhinav Bhatele. 2023. A hybrid tensor-expert-data parallelism approach to optimize mixture-of-experts training. In *Proceedings of the 37th International Conference on Supercomputing*. 203–214.

[169] Andrii Skliar, Ties van Rozendaal, Romain Lepert, Todor Boinovski, Mart van Baalen, Markus Nagel, Paul Whatmough, and Babak Ehteshami Bejnordi. 2025. Mixture of cache-conditional experts for efficient mobile device inference. *Transactions on Machine Learning Research*. https://openreview.net/forumid=ul4W26KEKz

[170] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. 1631–1642.

[171] Xiaoniu Song, Zihang Zhong, and Rong Chen. 2024. ProMoE: Fast MoE-based LLM serving using proactive caching. arXiv:2410.22134. Retrieved from https://arxiv.org/abs/2410.22134 (2024).

[172] Andrew Steane. 1998. Quantum computing. *Reports on Progress in Physics* 61, 2 (1998), 117.

[173] Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Daniel Li, Wen-tau Yih, Jason Weston, et al. 2024. Branch-Train-MiX: Mixing expert LLMs into a mixture-of-experts LLM. *First Conference on Language Modeling*. https://openreview.net/forum?id=nqLAuMOF6n

[174] Mengshu Sun, Haoyu Ma, Guoliang Kang, Yifan Jiang, Tianlong Chen, Xiaolong Ma, Zhangyang Wang, and Yanzhi Wang. 2022. Vaqf: Fully automatic software-hardware co-design framework for low-bit vision transformer. arXiv:2201.06618. Retrieved from https://arxiv.org/abs/2201.06618 (2022).

[175] Xingwu Sun, Yanfeng Chen, Yiqing Huang, Ruobing Xie, Jiaqi Zhu, Kai Zhang, Shuaipeng Li, Zhen Yang, Jonny Han, Xiaobo Shu, et al. 2024. Hunyuan-large: An open-source MoE model with 52 Billion activated parameters by Tencent. arXiv:2411.02265. Retrieved from https://arxiv.org/abs/2411.02265 (2024).

[176] Shawn Tan, Yikang Shen, Zhenfang Chen, Aaron Courville, and Chuang Gan. 2023. Sparse universal transformer. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 169–179.

[177] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. HOBBIT: A mixed precision expert offloading system for fast MoE inference. arXiv:2411.01433. Retrieved from https://arxiv.org/abs/2411.01433 (2024).

[178] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model. (2023).

[179] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530. Retrieved from https://arxiv.org/abs/2403.05530 (2024).

[180] Google Brain Team. 2024. TensorFlow. Retrieved from http://tensorflow.org/

[181] Meituan LongCat Team, Bei Li, Bingye Lei, Bo Wang, Bolin Rong, Chao Wang, Chao Zhang, Chen Gao, Chen Zhang, Cheng Sun, et al. 2025. LongCat-flash technical report. arXiv:2509.01322. Retrieved from https://arxiv.org/abs/2509.01322 (2025).

[182] Qwen Team. 2024. Qwen1.5-MoE: Matching 7B Model Performance with 1/3 Activated Parameters. Retrieved from https://qwenlm.github.io/blog/qwen-moe/

[183] The Mosaic Research Team. 2024. Introducing DBRX: A New State-of-the-Art Open LLM. Retrieved from https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm

[184] Tencent Hunyuan Team. 2025. Hunyuan-A13B Technical Report. Retrieved October 20, 2025 from https://github.com/Tencent-Hunyuan/Hunyuan-A13B/blob/main/report/Hunyuan_A13B_Technical_Report.pdf

[185] Volker Tresp. 2000. Mixtures of gaussian processes. In *Proceedings of the Advances in Neural Information Processing Systems*, T. Leen, T. Dietterich, and V. Tresp (Eds.), Vol. 13. MIT Press.

[186] Marcos Treviso, Ji-Ung Lee, Tianchu Ji, Betty van Aken, Qingqing Cao, Manuel R. Ciosici, Michael Hassid, Kenneth Heafield, Sara Hooker, Colin Raffel, et al. 2023. Efficient methods for natural language processing: A survey. *Transactions of the Association for Computational Linguistics* 11 (2023), 826–860.

[187] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017).

[188] Arpita Vats, Rahul Raja, Vinija Jain, and Aman Chadha. 2024. The evolution of mixture of experts: A survey from basics to breakthroughs. *Preprints* (August 2024). DOI: https://doi.org/10.20944/preprints202408.0583.v2

[189] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, et al. 2024. Efficient large language models: A survey. *Transactions on Machine Learning Research*. https://openreview.net/forumid=bsCCJHbO8A

[190] Alex Wang. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. 353–355.

[191] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. Bitnet: Scaling 1-bit transformers for large language models. arXiv:2310.11453. Retrieved from https://arxiv.org/abs/2310.11453 (2023).

[192] Hongyi Wang, Felipe Maia Polo, Yuekai Sun, Souvik Kundu, Eric Xing, and Mikhail Yurochkin. 2024. Fusing models with complementary expertise. *The Twelfth International Conference on Learning Representations*. https://openreview.net/forumid=PhMrGCMIRL

[193] Peng Wang, An Yang, Rui Men, Junyang Lin, Shuai Bai, Zhikang Li, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022. Ofa: Unifying architectures, tasks, and modalities through a simple sequence-to-sequence learning framework. In *Proceedings of the International Conference on Machine Learning*. PMLR, 23318–23340.

[194] Wei Wang, Zhiquan Lai, Shengwei Li, Weijie Liu, Keshi Ge, Yujie Liu, Ao Shen, and Dongsheng Li. 2023. Prophet: Fine-grained Load Balancing for Parallel Training of Large-scale MoE Models. In *Proceedings of the 2023 IEEE International Conference on Cluster Computing (CLUSTER)*. 82–94. DOI: https://doi.org/10.1109/CLUSTER52292.2023.00015

[195] Xin Wang, Fisher Yu, Lisa Dunlap, Yi-An Ma, Ruth Wang, Azalia Mirhoseini, Trevor Darrell, and Joseph E. Gonzalez. 2020. Deep mixture of experts via shallow embedding. In *Proceedings of the Uncertainty in Artificial Intelligence*. PMLR, 552–562.

[196] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. 2025. Svd-llm: Truncation-aware singular value decomposition for large language model compression. *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forumid=LNYIUouhdt

[197] Tianwen Wei, Bo Zhu, Liang Zhao, Cheng Cheng, Biye Li, Weiwei Lü, Peng Cheng, Jianhao Zhang, Xiaoyu Zhang, Liang Zeng, et al. 2024. Skywork-MoE: A deep dive into training techniques for mixture-of-experts language models. arXiv:2406.06563. Retrieved from https://arxiv.org/abs/2406.06563 (2024).

[198] Shaohua Wu, Jiangang Luo, Xi Chen, Lingjun Li, Xudong Zhao, Tong Yu, Chao Wang, Yue Wang, Fei Wang, Weixu Qiao, et al. 2024. Yuan 2.0-M32: Mixture of experts with attention router. arXiv:2405.17976. Retrieved from https://arxiv.org/abs/2405.17976 (2024).

[199] Yongji Wu, Wenjie Qu, Tianyang Tao, Zhuang Wang, Wei Bai, Zhuohao Li, Yuan Tian, Jiaheng Zhang, Matthew Lentz, and Danyang Zhuo. 2024. Lazarus: Resilient and elastic training of mixture-of-experts models with adaptive expert placement. arXiv:2407.04656. Retrieved from https://arxiv.org/abs/2407.04656 (2024).

[200] xAI. 2024. Open Release of Grok-1. Retrieved from https://x.ai/blog/grok-os

[201] Xinfeng Xia, Jiacheng Liu, Xiaofeng Hou, Peng Tang, Mingxuan Zhang, Wenfeng Wang, and Chao Li. 2025. MoE-Prism: Disentangling monolithic experts for elastic MoE services via model-system co-designs. arXiv:2510.19366. Retrieved from https://arxiv.org/abs/2510.19366 (2025).

[202] Yanyue Xie, Zhi Zhang, Ding Zhou, Cong Xie, Ziang Song, Xin Liu, Yanzhi Wang, Xue Lin, and An Xu. 2024. MoE-pruner: Pruning mixture-of-experts large language model using the hints from its router. arXiv:2410.12013. Retrieved from https://arxiv.org/abs/2410.12013 (2024).

[203] Deyi Xiong and Zhiyuan Zeng. 2023. SCoMoE: Efficient mixtures of experts with structured communication. In *Proceedings of the 11th International Conference on Learning Representations*.

[204] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, Daliang Xu, Qipeng Wang, Bingyang Wu, Yihao Zhao, Chen Yang, Shihe Wang, et al. 2024. A survey of resource-efficient llm and multimodal foundation models. arXiv:2401.08092. Retrieved from https://arxiv.org/abs/2401.08092 (2024).

[205] Fuzhao Xue, Xiaoxin He, Xiaozhe Ren, Yuxuan Lou, and Yang You. 2022. One student knows all experts know: From sparse to dense. arXiv:2201.10890. Retrieved from https://arxiv.org/abs/2201.10890 (2022).

[206] Fuzhao Xue, Zian Zheng, Yao Fu, Jinjie Ni, Zangwei Zheng, Wangchunshu Zhou, and Yang You. 2024. Openmoe: An early effort on open mixture-of-experts language models. In *Proceedings of the 41st International Conference on Machine Learning*. 55625–55655.

[207] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. Moe-infinity: Activation-aware expert offloading for efficient moe serving. arXiv:2401.14361. Retrieved from https://arxiv.org/abs/2401.14361 (2024).

[208] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. arXiv:2505.09388. Retrieved from https://arxiv.org/abs/2505.09388 (2025).

[209] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. arXiv:2407.10671. Retrieved from https://arxiv.org/abs/2407.10671 (2024).

[210] Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Yuanlin Duan, Wenqi Jia, Miao Yin, Yu Cheng, and Bo Yuan. 2024. MoE-I2: Compressing mixture of experts models through inter-expert pruning and intra-expert low-rank decomposition. *Findings of the Association for Computational Linguistics: EMNLP 2024*. 10456–10466.

[211] Yuanhang Yang, Shiyi Qi, Wenchao Gu, Chaozheng Wang, Cuiyun Gao, and Zenglin Xu. 2024. XMoE: Sparse models with fine-grained and adaptive expert selection. In *Proceedings of the Findings of the Association for Computational Linguistics ACL 2024*. 11664–11674.

[212] Yi Yang, Wen-tau Yih, and Christopher Meek. 2015. Wikiqa: A challenge dataset for open-domain question answering. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. 2013–2018.

[213] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2369–2380.

[214] Jinghan Yao, Quentin Anthony, Aamir Shafi, Hari Subramoni, and Dhabaleswar K. D. K. Panda. 2024. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. In *Proceedings of the 2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 915–925.

[215] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2025. EdgeMoE: Empowering sparse large language models on mobile devices. *IEEE Transactions on Mobile Computing* 24, 8 (2025), 7059–7073.

[216] Haoran You, Zhanyi Sun, Huihong Shi, Zhongzhi Yu, Yang Zhao, Yongan Zhang, Chaojian Li, Baopu Li, and Yingyan Lin. 2023. Vitcod: Vision transformer acceleration via dedicated algorithm and accelerator co-design. In *Proceedings of the 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 273–286.

[217] Dianhai Yu, Liang Shen, Hongxiang Hao, Weibao Gong, Huachao Wu, Jiang Bian, Lirong Dai, and Haoyi Xiong. 2024. MoESys: A distributed and efficient mixture-of-experts training and inference system for internet services. *IEEE Transactions on Services Computing* 17, 5 (2024), 2626–2639. DOI: https://doi.org/10.1109/TSC.2024.3399654

[218] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2024. Metamath: Bootstrap your own mathematical questions for large language models. *The Twelfth International Conference on Learning Representations*. https://openreview.net/forumid=N8N0hgNDRt

[219] Xiaoming Yuan, Weixuan Kong, Zhenyu Luo, and Minrui Xu. 2024. Efficient inference offloading for mixture-of-experts large language models in internet of medical things. *Electronics* 13, 11 (2024), 2077.

[220] Yuping Yuan, Zhao You, Shulin Feng, Dan Su, Yanchun Liang, Xiaohu Shi, and Dong Yu. 2023. Compressed MoE ASR model based on knowledge distillation and quantization. *INTERSPEECH*. 3337–3341.

[221] Zhihang Yuan, Yuzhang Shang, Yue Song, Qiang Wu, Yan Yan, and Guangyu Sun. 2023. Asvd: Activation-aware singular value decomposition for compressing large language models. arXiv:2312.05821. Retrieved from https://arxiv.org/abs/2312.05821 (2023).

[222] Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, et al. 2024. Llm inference unveiled: Survey and roofline model insights. arXiv:2402.16363. Retrieved from https://arxiv.org/abs/2402.16363 (2024).

[223] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching. *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1429–1443.

[224] Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, et al. 2025. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. arXiv:2508.06471. Retrieved from https://arxiv.org/abs/2508.06471 (2025).

[225] Mingshu Zhai, Jiaao He, Zixuan Ma, Zan Zong, Runqing Zhang, and Jidong Zhai. 2023. SmartMoE: Efficiently training sparsely-activated models through combining offline and online parallelization. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 961–975. Retrieved from https://www.usenix.org/conference/atc23/presentation/zhai

[226] Qizhen Zhang, Nikolas Gritsch, Dwaraknath Gnaneshwar, Simon Guo, David Cairuz, Bharat Venkitesh, Jakob Foerster, Phil Blunsom, Sebastian Ruder, Ahmet Ustun, et al. 2024. BAM! just like that: Simple and efficient parameter upcycling for mixture of experts. *Advances in Neural Information Processing Systems* 37 (2024), 56304–56321.

[227] Shiwei Zhang, Lansong Diao, Chuan Wu, Siyu Wang, and Wei Lin. 2022. Accelerating large-scale distributed neural network training with SPMD parallelism. In *Proceedings of the 13th Symposium on Cloud Computing*. 403–418.

[228] Xiaofeng Zhang, Yikang Shen, Zeyu Huang, Jie Zhou, Wenge Rong, and Zhang Xiong. 2022. Mixture of attention heads: Selecting attention heads per token. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 4150–4162.

[229] Zeliang Zhang, Xiaodong Liu, Hao Cheng, Chenliang Xu, and Jianfeng Gao. 2025. Diversifying the expert knowledge for task-agnostic pruning in sparse mixture-of-experts. *Findings of the Association for Computational Linguistics: ACL 2025*. 86–102.

[230] Zixiao Zhang, Xiaoqiang Lu, Guojin Cao, Yuting Yang, Licheng Jiao, and Fang Liu. 2021. ViT-YOLO: Transformer-based YOLO for object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2799–2808.

[231] Zheng Zhang, Yaqi Xia, Hulin Wang, Donglin Yang, Chuang Hu, Xiaobo Zhou, and Dazhao Cheng. 2024. MPMoE: Memory efficient MoE for pre-trained models with adaptive pipeline parallelism. *IEEE Transactions on Parallel and Distributed Systems* 35, 6 (2024), 998–1011. DOI: https://doi.org/10.1109/TPDS.2024.3385639

[232] Hao Zhao, Zihan Qiu, Huijia Wu, Zili Wang, Zhaofeng He, and Jie Fu. 2024. HyperMoE: Towards better mixture of experts via transferring among experts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10605–10618. DOI: https://doi.org/10.18653/v1/2024.acl-long.571

[233] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, et al. 2022. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 559–578. Retrieved from https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin

[234] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.

[235] Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. 2024. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems* 36 (2024), 55006–55021.

[236] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A memory-based acceleration via software-hardware co-design for transformer. In *Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 1071–1085. DOI: https://doi.org/10.1109/HPCA53966.2022.00082

[237] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Quoc V. Le, and James Laudon. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.

[238] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Quoc V. Le, James Laudon, Zhifeng Chen. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.

[239] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. 2024. A survey on efficient inference for large language models. arXiv:2404.14294. Retrieved from https://arxiv.org/abs/2404.14294 (2024).

[240] Hanqing Zhu, Jiaqi Gu, Hanrui Wang, Zixuan Jiang, Zhekai Zhang, Rongxing Tang, Chenghao Feng, Song Han, Ray T. Chen, and David Z. Pan. 2024. Lightening-transformer: A dynamically-operated optically-interconnected photonic transformer accelerator. In *Proceedings of the 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 686–703.

[241] Tong Zhu, Xiaoye Qu, Daize Dong, Jiacheng Ruan, Jingqi Tong, Conghui He, and Yu Cheng. 2024. Llama-moe: Building mixture-of-experts from llama with continual pre-training. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 15913–15923.

[242] Yan Zhuang, Zhenzhe Zheng, Fan Wu, and Guihai Chen. 2024. LiteMoE: Customizing on-device LLM serving via proxy submodel tuning. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems*. 521–534.

Received 21 January 2025; revised 5 November 2025; accepted 8 January 2026